

Unit-IV: Advanced Modeling and Civil Engineering Applications

15 hr | Subject Code: 24CV201T | ODD SEM 2026-27

Dr. Ankit Deshmukh

2026-10-06

Contents

1 What You Will Learn in Unit 4	4
2 The Unit 4 Roadmap	5
2.1 Hour Plan (14 Hours)	5
3 Regression Models	6
3.1 Linear Regression for Structural Design	6
3.1.1 The idea in one line	6
3.1.2 Simulate a concrete mix dataset	6
3.1.3 Fit and visualize	7
3.1.4 Always check residuals (Unit 3 pays off!)	7
3.1.5 Multiple regression: add cement content and age	8
3.1.6 Predict a new mix (with uncertainty)	10
3.2 Logistic Regression for Failure Prediction	11
3.2.1 The idea in one line	11
3.2.2 Simulate a water-pipeline failure dataset	12
3.2.3 Fit the model and interpret coefficients	12
3.2.4 The decision boundary on the age–pressure plane	13
3.2.5 Evaluation: confusion matrix, threshold tuning, ROC	15
4 Tree-Based Models	17
4.1 Decision Trees	17
4.1.1 The idea in one line	17
4.1.2 Simulate a pavement-management dataset	17
4.1.3 Train / test split and tree fitting	19
4.1.4 How a tree carves up feature space	19

4.1.5	Pruning: choosing tree size with the complexity parameter	21
4.1.6	Test performance	21
4.2	Random Forests	22
4.2.1	The idea in one line	22
4.2.2	Simulate a flood susceptibility dataset	23
4.2.3	Train the forest	24
4.2.4	Watch the OOB error converge as trees are added	25
4.2.5	Which factors drive flooding? Variable importance	25
4.2.6	Partial dependence: how does probability respond to one factor?	26
4.2.7	Test-set ROC	27
4.2.8	Bonus application: crop yield (regression forest)	28
5	Proximity and Probability Models	29
5.1	K-Nearest Neighbors (KNN)	29
5.1.1	The idea in one line	29
5.1.2	Simulate a soil classification dataset	29
5.1.3	See the algorithm “think”: neighborhoods of a new sample	30
5.1.4	Decision boundaries: the effect of k	32
5.1.5	Choosing k by validation	32
5.1.6	The scaling trap (very important!)	33
5.2	Naive Bayes	34
5.2.1	The idea in one line	34
5.2.2	Simulate a flood-day dataset	35
5.2.3	Fit, inspect, predict	36
6	Clustering Techniques for Regional Analysis	39
6.1	K-Means	39
6.1.1	The idea in one line	39
6.1.2	Simulate 360 settlements (urban / suburban / rural)	39
6.1.3	Choosing K: elbow method + silhouette	40
6.1.4	Final clustering and interpretation	41
6.2	Hierarchical Clustering	43
6.2.1	The idea in one line	43
6.2.2	Linkage matters: same data, four dendrograms	44

7	Geospatial Data Structures	46
7.1	Raster vs Vector — The Two Languages of Maps	46
7.2	Rasters in R with terra	48
7.2.1	Resolution matters — coarsening the mesh	49
7.3	Vectors in R with sf	49
8	Satellite Image Processing	51
8.0.1	Simulate a 4-band image (Red, Green, Blue, NIR)	51
8.0.2	NDVI — the most-used vegetation index on Earth	53
9	Remote Sensing Integration: GIS + AI Fusion	55
9.1	The Workflow That Runs Modern Geospatial ML	55
9.2	Full Demonstration: Flood Susceptibility Mapping End-to-End	57
9.2.1	Step 1 — build the geospatial knowledge base (4 terrain layers)	57
9.2.2	Step 2 — extract features at 400 training points (map → table)	57
9.2.3	Step 3 — train the model on the table	59
9.2.4	Step 4 — predict on the whole raster (table → map)	59
9.2.5	Step 5 — convert probabilities into planning zones	61
10	Advanced Mapping Applications	62
10.1	Urban Heat Island (UHI) Analysis	62
10.1.1	The transect: temperature along an east–west line	62
10.1.2	The green-cooling effect: NDVI vs LST	64
10.2	Landslide Susceptibility Mapping	64
10.2.1	Predict the whole terrain	66
10.2.2	Area budget per zone (decision support)	68
10.3	River Change Detection	69
10.3.1	Remote-sensing style change detection with NDVI differencing	70
11	One-Page Summary (Pin This!)	72
12	Mini Glossary	73
12.1	References	74

1 What You Will Learn in Unit 4

In Units 1-3 you learned *what* AI is, *how* to explore data, and *how* to evaluate models. **Unit 4 is where you finally build the models** — and you point them at the things civil engineers care about: concrete, pipes, pavements, floods, crops, heat, landslides, and rivers. Then you connect ML to **maps**, because almost every civil engineering decision has a *location*.

By the end of this handout, you will be able to:

1. **Fit and interpret regression models** — linear for continuous targets (concrete strength), logistic for binary outcomes (pipe failure). *(CO4)*
2. **Build tree-based models** — decision trees and random forests — and read their variable-importance and partial-dependence plots. *(CO4, CO5)*
3. **Apply proximity and probability classifiers** — KNN and Naive Bayes — and know when each is appropriate. *(CO4)*
4. **Cluster unlabeled data** — K-means and hierarchical — to create homogeneous zones for planning. *(CO4)*
5. **Work with geospatial data** — rasters vs. vectors in R with `terra` and `sf`. *(CO6)*
6. **Process satellite imagery** — build NDVI maps and vegetation masks. *(CO6)*
7. **Fuse GIS with AI** — the `extract()` → model → `predict()` workflow that powers every modern susceptibility map. *(CO6)*
8. **Deliver advanced mapping products** — Urban Heat Island maps, landslide susceptibility zones, and river change detection. *(CO6)*

Civil connection: Every algorithm in this unit is illustrated with a civil engineering dataset — concrete mixes, water pipelines, pavements, flood sites, crop yields, cities, slopes, and rivers. The tools are general; the problems are the ones you will meet on site.

How to use this handout: Read a section, then run its code chunk in RStudio (or knit the whole file). Every dataset is **simulated inside the document**, so everything knits reproducibly on any laptop — even offline. At the end of each section we tell you where real datasets live (Bhuvan, USGS, Copernicus, Kaggle) so you can swap in real data during project work.

How to read the outputs: Wherever a code chunk prints a table or a figure, the section immediately after it (marked with a purple bar like this one) explains what you are looking at, what it means, and what to watch out for.

2 The Unit 4 Roadmap

What we build in Unit 4

Red = regression | Blue = classification | Purple = clustering | Green = geospatial | Yellow = applications

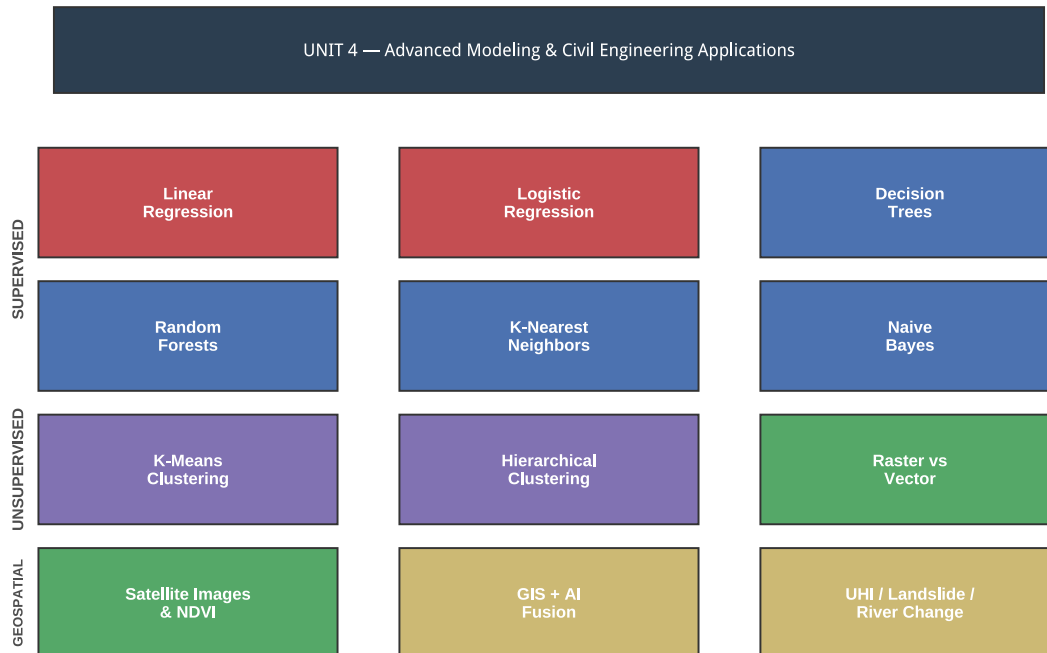


Figure 1: Unit 4 roadmap: every algorithm we learn, grouped by family.

How to read this figure: Each coloured box is one algorithm. The three vertical groups are the three ML families you will meet in this unit — **supervised** models (they need labeled examples), **unsupervised** models (they discover structure on their own), and **geospatial** tools (they connect models to maps). Red boxes solve regression problems; blue boxes solve classification problems; purple boxes group unlabeled data; green and yellow boxes carry us into the mapping half of the unit.

The master analogy for Unit 4: think of ML algorithms as a **site toolbox**. Regression = the *total station* (measures precise quantities). Trees and forests = the *site engineer's decision checklist*. KNN and Naive Bayes = asking your *most experienced mason* what he thinks. Clustering = *zoning the site* before planning. Geospatial tools = the *master plan drawing* that ties everything to location.

2.1 Hour Plan (14 Hours)

Topic	Hours	Section in this document
Regression models (linear + logistic)	2.5	Sections 2, 3
Tree-based models (decision trees, random forests)	3.0	Sections 4, 5
Proximity & probability models (KNN, Naive Bayes)	2.0	Sections 6, 7

Topic	Hours	Section in this document
Clustering (K-means, hierarchical)	2.0	Sections 8, 9
Geospatial data structures + satellite imagery	2.0	Sections 10, 11
GIS + AI fusion, UHI, landslide, river change	2.5	Sections 12-15

3 Regression Models

3.1 Linear Regression for Structural Design

3.1.1 The idea in one line

Fit the straight line $y = \beta_0 + \beta_1 x$ that minimizes the sum of squared vertical errors:

$$\min_{\beta_0, \beta_1} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

Civil analogy: You have done this physically already. When you draw the **initial tangent modulus** on a stress-strain curve, or a best-fit line through **load-deflection** readings of a beam test, you are fitting a linear regression by eye. The slope β_1 is a *stiffness-like* quantity; the residuals are your *measurement scatter*.

3.1.2 Simulate a concrete mix dataset

Physical truth we encode: **strength decreases as water-cement ratio increases** (Abrams' law), improves with cement content, and grows with age.

```
n <- 150
concrete <- tibble(
  mix_id = sprintf("M%03d", 1:n),
  wc      = round(runif(n, 0.38, 0.72), 3), # water-cement ratio
  cement  = round(runif(n, 300, 500), 0), # kg/m3
  age     = sample(c(7, 28, 90), n, replace = TRUE, prob = c(0.3, 0.5, 0.2))
)

age_fac <- case_when(
  concrete$age == 7 ~ 0.78,
  concrete$age == 28 ~ 1.00,
  concrete$age == 90 ~ 1.12
)

set.seed(11)
concrete <- concrete %>%
  mutate(strength = round(65 * age_fac * (cement / 400)^0.4 *
    exp(-1.8 * (wc - 0.45)) + rnorm(n, 0, 2.5), 1))

head(concrete)
```

```
## # A tibble: 6 x 5
##   mix_id    wc cement   age strength
##   <chr>   <dbl>   <dbl> <dbl>   <dbl>
## 1 M001    0.665    375    28    41.5
## 2 M002    0.489    389     7    46.8
## 3 M003    0.611    484    28    48.7
## 4 M004    0.617    336     7    31.6
## 5 M005    0.535    451    90    68.5
## 6 M006    0.618    387    28    45.1
```

What you are seeing: Each row is one concrete mix. `wc`, `cement`, and `age` are inputs; `strength` is the target. The last column is what we will try to predict. The simulation encodes Abrams' law (higher `wc` → lower strength), a mild cement effect, and age-dependence through `age_fac`.

3.1.3 Fit and visualize

```
fit1 <- lm(strength ~ wc, data = concrete)
b0 <- coef(fit1)[1]
b1 <- coef(fit1)[2]
r2 <- summary(fit1)$r.squared

ggplot(concrete, aes(wc, strength)) +
  geom_point(colour = "steelblue", alpha = 0.65, size = 2) +
  geom_smooth(method = "lm", se = TRUE, colour = "firebrick", fill = "pink") +
  annotate("text",
    x = 0.71, y = max(concrete$strength) - 1,
    label = sprintf("strength = %.1f %+.1f × w/c\nR² = %.2f", b0, b1, r2),
    hjust = 1, size = 4.2, colour = "grey20"
  ) +
  labs(
    title = "Simple linear regression: concrete strength vs w/c ratio",
    x = "Water-cement ratio (w/c)",
    y = "Compressive strength (MPa)"
  )
```

How to read this figure: Each blue dot is one mix. The red line is the fitted regression; the pink band is its 95% confidence interval. The annotation shows the equation and R^2 . **Reading the coefficients:** $\beta_1 \approx -60$ MPa per unit w/c. Because w/c moves in steps of 0.1, each 0.1 increase in w/c costs roughly **6 MPa** — exactly the rule of thumb used in mix-design labs. R^2 near 0.6–0.7 would be typical here: w/c alone explains most, but not all, of the strength variation.

3.1.4 Always check residuals (Unit 3 pays off!)

```
par(mfrow = c(2, 2))
plot(fit1, pch = 19, col = "steelblue", cex = 0.7)
```

```
par(mfrow = c(1, 1))
```

What to look for:

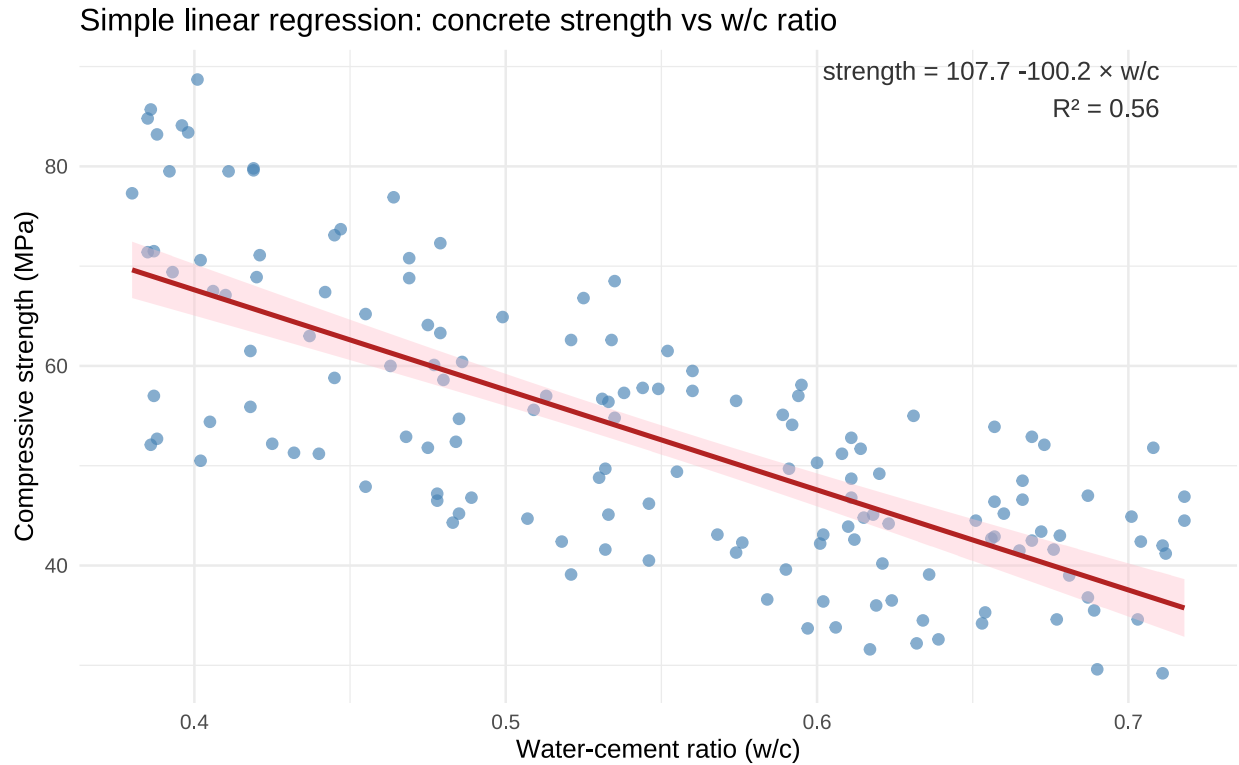


Figure 2: Linear regression: strength falls as w/c rises — Abrams' law recovered from data.

- *Residuals vs Fitted* — should look like a random cloud (no funnel, no curve). A curve here means the relationship is not really linear.
- *Normal Q-Q* — points should hug the diagonal. Strong departures mean the errors are not normal, which weakens confidence intervals.
- *Scale-Location* — no growing spread (heteroscedasticity). A funnel shape means the model is less reliable at high strengths.
- *Residuals vs Leverage* — watch for influential mixes (bad lab data). Points in the top-right corner can single-handedly tilt the whole line.

3.1.5 Multiple regression: add cement content and age

```
fit2 <- lm(strength ~ wc + cement + factor(age), data = concrete)
summary(fit2)$r.squared
```

```
## [1] 0.9538019
```

How to read this output: The single number printed is the **multiple R^2** — the fraction of strength variance explained by *all three* predictors together. You should see it climb noticeably above the simple-regression R^2 . That jump is the payoff of adding features that carry real physical information: cement content and curing age both matter, and the model uses them.

```
concrete$fitted2 <- fitted(fit2)
ggplot(concrete, aes(fitted2, strength, colour = factor(age))) +
```

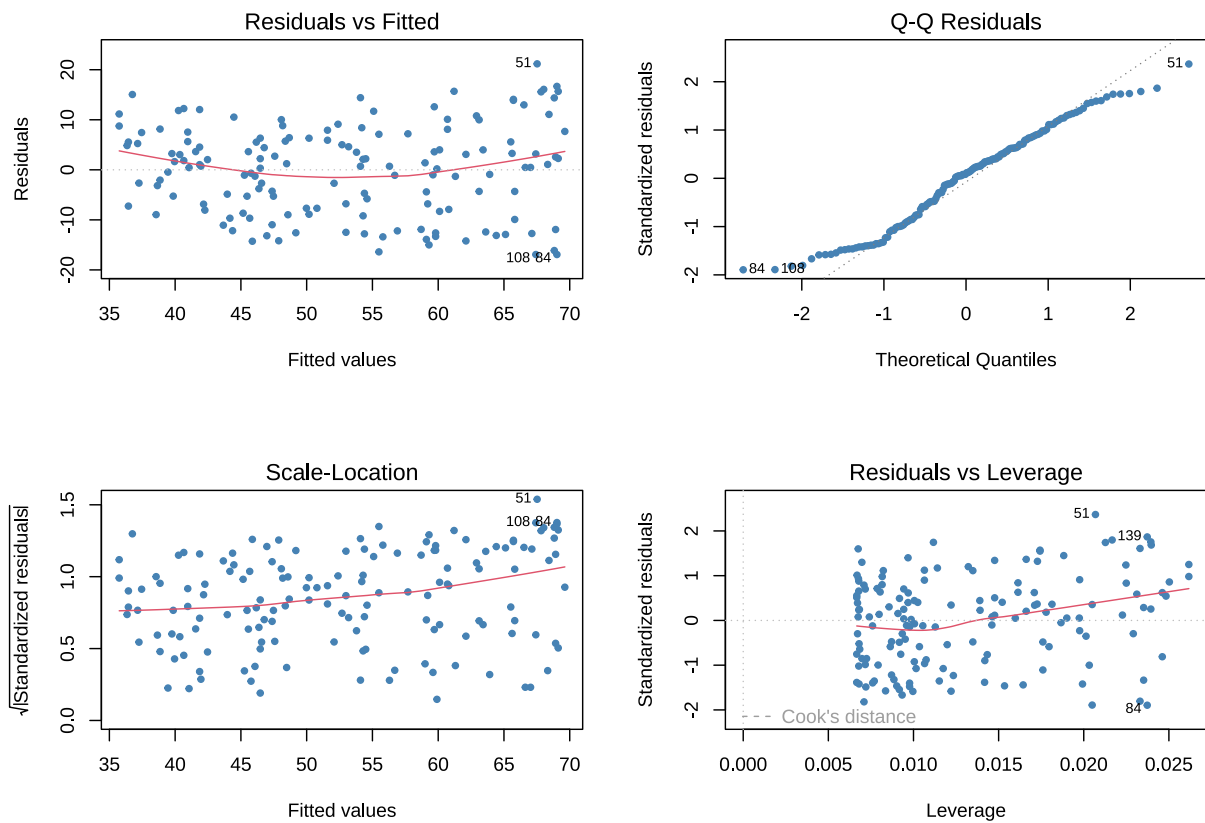


Figure 3: The four classic diagnostic plots from Unit 3, now applied to a structural model.

```
geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour = "grey40") +
geom_point(size = 2, alpha = 0.75) +
scale_colour_viridis_d(name = "Age (days)") +
labs(
  title = "Multiple regression fits better than simple regression",
  x = "Predicted strength (MPa)", y = "Observed strength (MPa)"
) +
coord_equal()
```

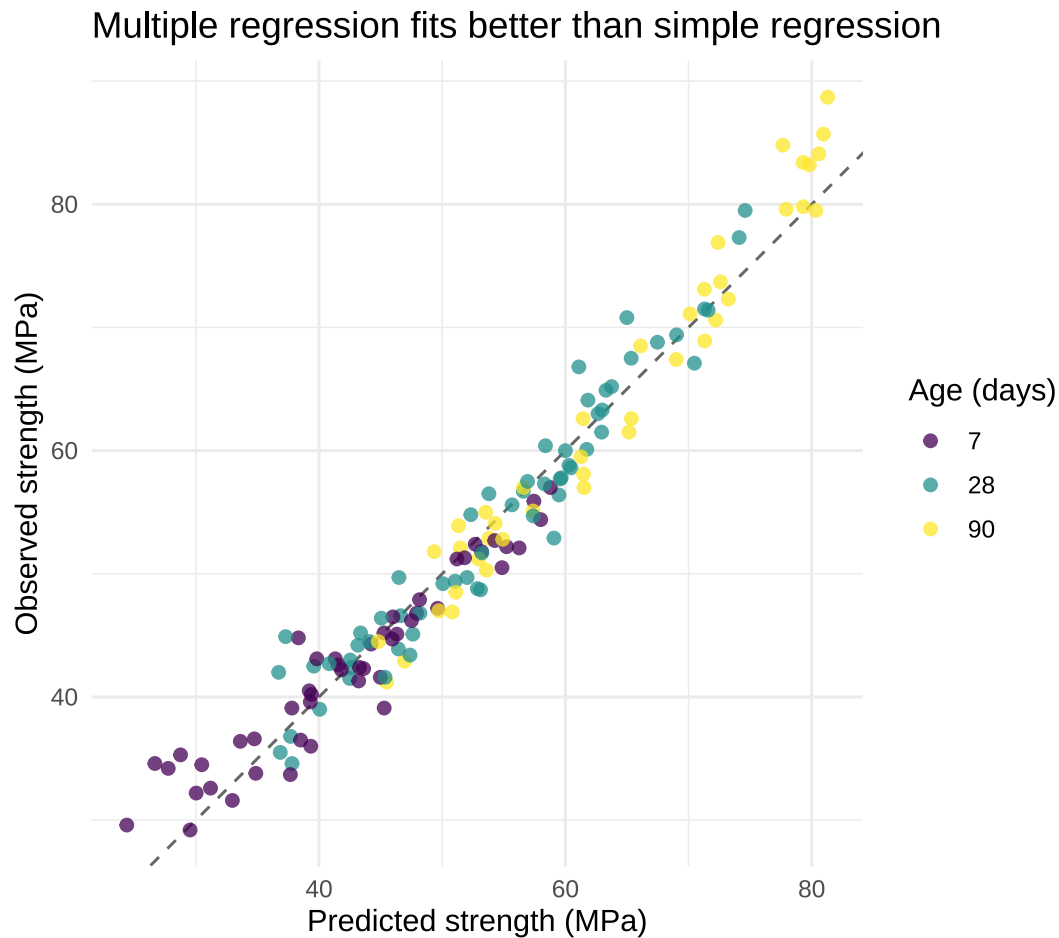


Figure 4: Observed vs fitted strength, coloured by curing age — points hug the 1:1 line.

How to read this figure: The dashed diagonal is the perfect-prediction line $y = x$. A good model puts points tightly around it. Compare the scatter here to what you would see from the simple model — the tighter and more symmetric the cloud, the better the fit. Colours let you check whether the model is systematically wrong for a particular age group (e.g., always under-predicting 7-day cubes).

3.1.6 Predict a new mix (with uncertainty)

```
new_mix <- data.frame(
  wc = 0.45, cement = 420,
```

```
age = factor(28, levels = sort(unique(concrete$age)))
)
predict(fit2, new_mix, interval = "predict")
```

```
##      fit      lwr      upr
## 1 65.4311 59.53483 71.32738
```

How to read this output: The `fit` column is the point prediction of 28-day strength for a mix with $w/c = 0.45$ and 420 kg/m^3 of cement. The `lwr`–`upr` pair is the **prediction interval** — the range in which a *single future cube* from this mix will fall with 95% probability. It is much wider than a confidence interval for the mean, because batch-to-batch variability is included. This is the number a careful engineer quotes to the client.

Where linear regression appears in CE practice: concrete strength, settlement vs. log-time, beam deflection vs. span (linear in the coefficients), trip generation vs. floor area, flood discharge vs. catchment area (rating curves).

3.2 Logistic Regression for Failure Prediction

3.2.1 The idea in one line

We want a **probability** of failure, not a number. Logistic regression passes the familiar linear score through an S-curve:

$$p = P(y = 1) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$$

Civil analogy: A *factor of safety* does not tell you “the structure fails or not” — it tells you how *close* you are. The sigmoid is the statistical factor-of-safety: it converts a linear “danger score” into a probability between 0 and 1.

```
z <- seq(-8, 8, length.out = 400)
tibble(z = z, p = plogis(z)) %>%
  ggplot(aes(z, p)) +
  geom_line(colour = "firebrick", linewidth = 1.3) +
  geom_hline(yintercept = 0.5, linetype = "dashed", colour = "grey40") +
  annotate("text", x = 5.4, y = 0.56, label = "decision threshold\np = 0.5", size = 3.4, colour = "grey40") +
  annotate("text", x = -4.6, y = 0.18, label = "z very negative\n→ 'asset healthy'", size = 3.4, colour = "green") +
  annotate("text", x = 4.6, y = 0.82, label = "z very positive\n→ 'asset failing'", size = 3.4, colour = "red") +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.25)) +
  labs(
    x = "z = 0 + 1·age + 2·pressure (linear danger score)",
    y = "P(failure)", title = "The sigmoid function"
  )
```

How to read this figure: The horizontal axis is the linear score z (however big or small the “danger” signal is). The vertical axis is the resulting **probability**. Notice that no matter how extreme z is, the output is always between 0 and 1 — that is what a probability must do. The dashed line at 0.5 is only a *default* cutoff; Section 3 will show you how to move it.



Figure 5: The sigmoid squashes any real number into a probability. The dashed line is the 0.5 decision threshold.

3.2.2 Simulate a water-pipeline failure dataset

```
set.seed(21)
n_pipe <- 400
pipe <- tibble(
  age      = round(runif(n_pipe, 1, 40), 1), # years in service
  pressure = round(runif(n_pipe, 1, 6), 2) # average operating pressure (bar)
) %>%
  mutate(
    logit   = -6 + 0.08 * age + 0.55 * pressure,
    prob    = plogis(logit),
    failure = rbinom(n_pipe, 1, prob) # 1 = burst/leak recorded
  )
mean(pipe$failure)
```

```
## [1] 0.135
```

What you are seeing: The last printed number is the **base rate** of failure in the dataset. It should be well below 0.5 — pipe failures are rare events. That matters because it drives which metric you trust later (Unit 3 lesson: recall over accuracy for rare events).

3.2.3 Fit the model and interpret coefficients

```
fit_logit <- glm(failure ~ age + pressure, data = pipe, family = binomial)

tibble(
  term      = names(coef(fit_logit)),
  beta      = round(coef(fit_logit), 4),
  odds_ratio = round(exp(coef(fit_logit)), 3)
)
```

```
## # A tibble: 3 x 3
##   term      beta odds_ratio
##   <chr>    <dbl>    <dbl>
## 1 (Intercept) -5.20      0.006
## 2 age         0.0603     1.06
## 3 pressure    0.496     1.64
```

How to read an odds ratio: e^β . For age, $e^{0.08} \approx 1.08$: every extra service year **multiplies the odds of failure by 1.08** (8% growth per year, compounding). This is exactly how maintenance engineers talk about *deterioration rates*. For pressure, $e^{0.55} \approx 1.73$: each extra bar of operating pressure multiplies failure odds by ~ 1.73 — a very strong effect, as pipe engineers expect.

3.2.4 The decision boundary on the age–pressure plane

```
grid <- expand.grid(
  age = seq(0, 40, length.out = 160),
  pressure = seq(1, 6, length.out = 160)
)
grid$prob <- predict(fit_logit, newdata = grid, type = "response")

ggplot() +
  geom_raster(data = grid, aes(age, pressure, fill = prob)) +
  scale_fill_viridis_c(name = "P(failure)") +
  geom_point(
    data = pipe, aes(age, pressure, colour = factor(failure)),
    size = 1.6, alpha = 0.9
  ) +
  scale_colour_manual(
    values = c("0" = "grey20", "1" = "red"),
    labels = c("No failure", "Failure"), name = "Observed"
  ) +
  labs(
    title = "Pipeline failure model: probability landscape",
    x = "Pipe age (years)", y = "Operating pressure (bar)"
  )
)
```

How to read this figure: The background colour is the model's *predicted* probability of failure at every (age, pressure) combination. Dark purple = safe; yellow = dangerous. Overlaid points are the real observed outcomes — black = no failure, red = failure. A good model puts red dots in yellow regions and black dots in purple regions. The **white diagonal** is the 0.5 contour — a practical safe-operating envelope. Anything above-right of it needs maintenance priority.

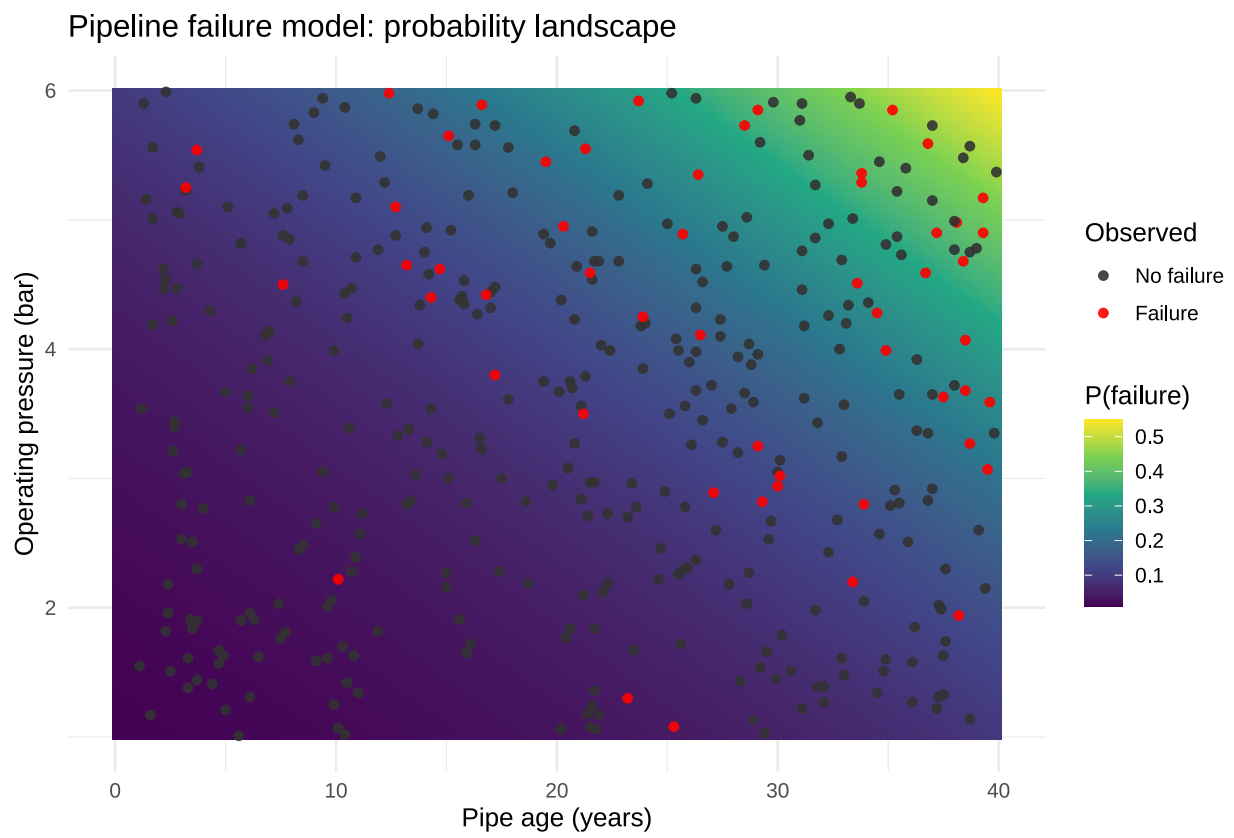


Figure 6: Background colour = model-predicted failure probability. The white region is 'safe operating envelope'.

3.2.5 Evaluation: confusion matrix, threshold tuning, ROC

```
prob_pred <- predict(fit_logit, type = "response")
pred_class <- ifelse(prob_pred > 0.5, 1, 0)
cm <- table(Predicted = pred_class, Actual = pipe$failure)

as.data.frame(cm) %>%
  ggplot(aes(Actual, Predicted, fill = Freq)) +
  geom_tile(colour = "white", linewidth = 2) +
  geom_text(aes(label = Freq), size = 7, fontface = "bold") +
  scale_fill_viridis_c(option = "mako", direction = -1) +
  labs(title = "Confusion matrix - pipeline failures")
```

Confusion matrix — pipeline failures

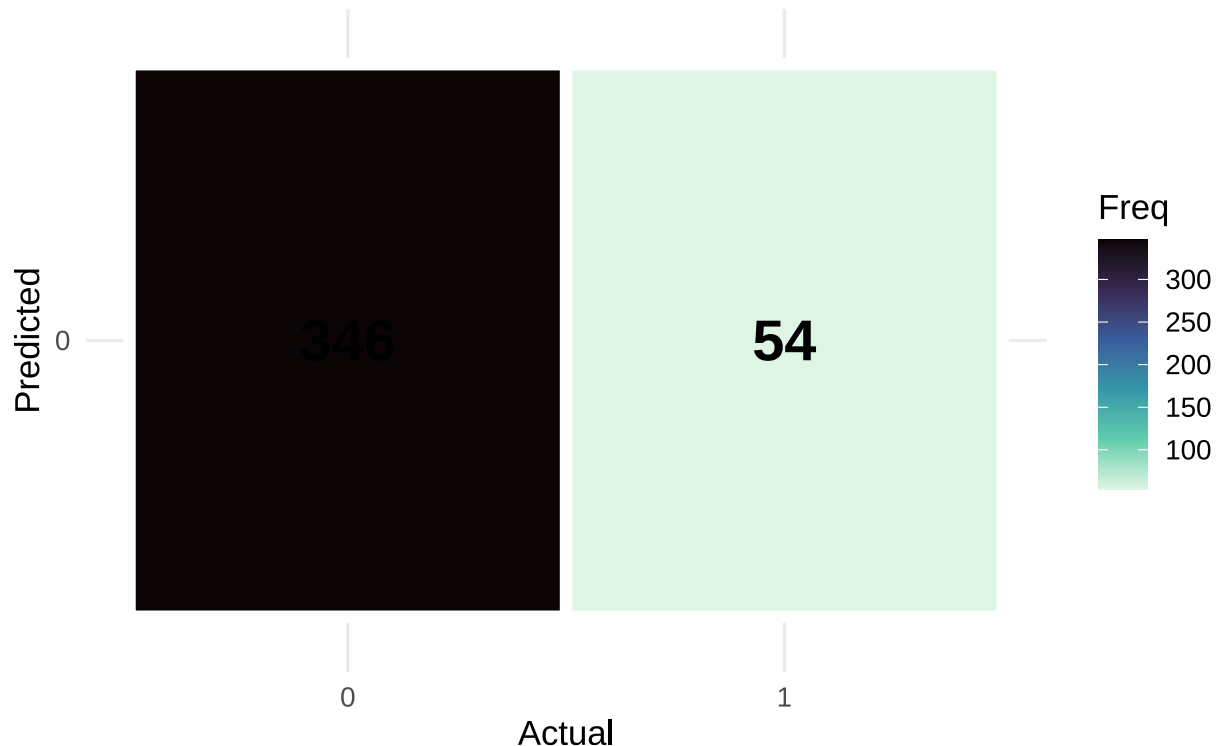


Figure 7: Confusion matrix at the default 0.5 threshold.

How to read the confusion matrix: Each cell counts cases. The top-left cell is True Negative (we said “no failure”, and there was none). The bottom-right is True Positive (we predicted failure, it failed — good catch). The bottom-left is False Negative — a **missed failure**, which in a water utility means an unplanned burst. The top-right is a False Positive — a maintenance truck sent out for nothing.

```
metrics <- purrr::map_dfr(seq(0.05, 0.95, 0.01), function(t) {
  pred <- as.integer(prob_pred > t)
  tp <- sum(pred == 1 & pipe$failure == 1)
  fp <- sum(pred == 1 & pipe$failure == 0)
  fn <- sum(pred == 0 & pipe$failure == 1)
})
```

```

tibble(
  threshold = t,
  accuracy = mean(pred == pipe$failure),
  precision = tp / max(tp + fp, 1),
  recall = tp / max(tp + fn, 1)
)
})

metrics %>%
  pivot_longer(-threshold) %>%
  ggplot(aes(threshold, value, colour = name)) +
  geom_line(linewidth = 1.1) +
  labs(
    title = "Threshold tuning: no single threshold wins everything",
    x = "Probability threshold", y = "Metric value", colour = NULL
  )

```

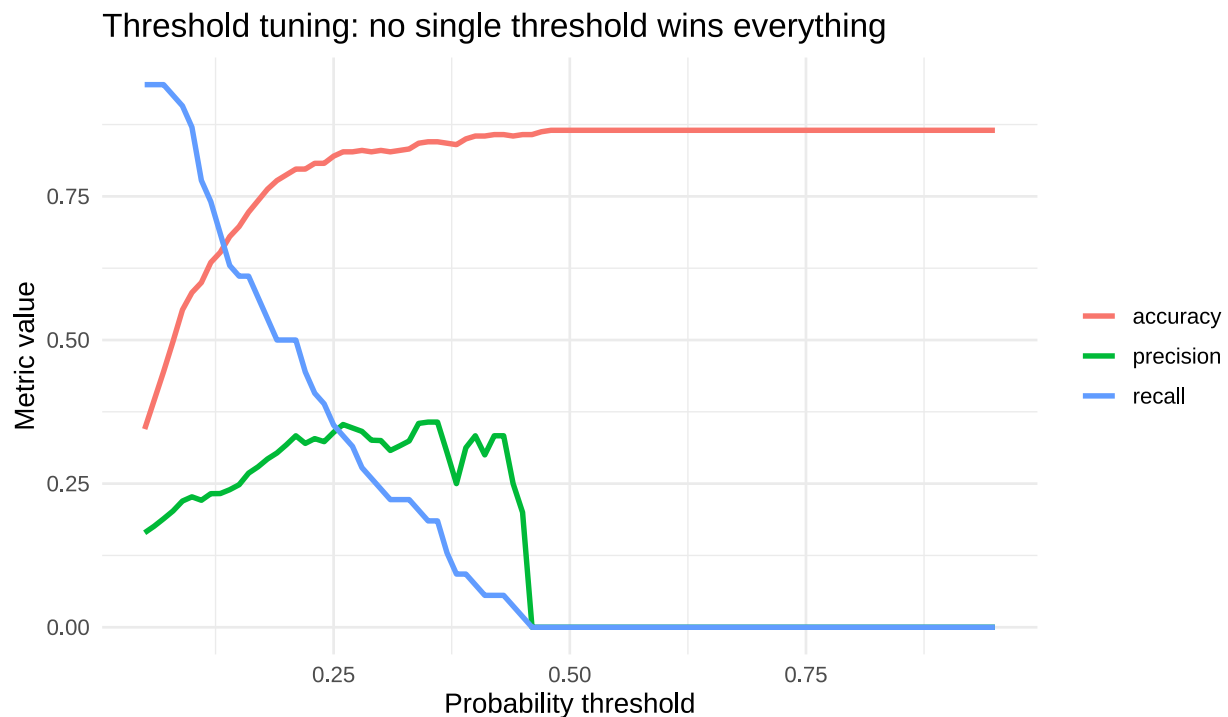


Figure 8: Moving the threshold trades precision against recall — the water board chooses based on repair budget.

How to read this figure: As the threshold moves left (more warnings), **recall rises** (we catch more real failures) but **precision falls** (more false alarms). As it moves right, the reverse. Accuracy is a misleading flat curve — for rare events it barely moves, because most cases are non-failures regardless. The engineering decision is which trade-off you can afford: a utility with a large repair crew can tolerate low precision; one with a small crew cannot.

```

roc_fit <- roc(pipe$failure, prob_pred, quiet = TRUE)
roc_df <- tibble(
  fpr = 1 - roc_fit$specificities,

```

```

tpr = roc_fit$sensitivities
) %>% arrange(fpr, tpr)

ggplot(roc_df, aes(fpr, tpr)) +
  geom_path(colour = "firebrick", linewidth = 1.2) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour = "grey50") +
  annotate("text",
    x = 0.62, y = 0.12, size = 5.5, fontface = "bold",
    label = sprintf("AUC = %.3f", auc(roc_fit))
  ) +
  coord_equal() +
  labs(
    title = "ROC - logistic failure model",
    x = "False positive rate", y = "True positive rate"
  )

```

How to read the ROC curve: The horizontal axis is the false-alarm rate; the vertical is the true-positive rate. The dashed diagonal is a useless model. The red curve bulges upward — the closer to the top-left corner, the better. AUC of ~0.85 is a strong utility-grade classifier; the printed value in your run will land in that range.

Real CE uses of logistic regression: pipe burst prediction (water utilities worldwide), pavement failure within analysis period, slope failure yes/no, contractor bid win/loss, defect presence in bridge inspection records.

4 Tree-Based Models

4.1 Decision Trees

4.1.1 The idea in one line

Repeatedly split the data with yes/no questions, choosing the split that makes groups **purer**. Purity is measured by **Gini impurity** $G = 1 - \sum_k p_k^2$ or **entropy** $H = -\sum_k p_k \log_2 p_k$.

Civil analogy: A decision tree is literally the **maintenance flowchart** in your inspection manual: “Is IRI > 4.5? → Is rutting > 10 mm? → Overlay vs. Reconstruction.” ML discovers the flowchart from data instead of from committee meetings.

4.1.2 Simulate a pavement-management dataset

```

set.seed(31)
n_sec <- 400
pavement <- tibble(
  section = sprintf("S%04d", 1:n_sec),
  age      = round(runif(n_sec, 1, 25), 1), # years since construction
  traffic  = round(runif(n_sec, 0.2, 12), 2), # million ESALs / year
  rut      = round(runif(n_sec, 2, 14), 1) # mm rut depth
) %>%

```

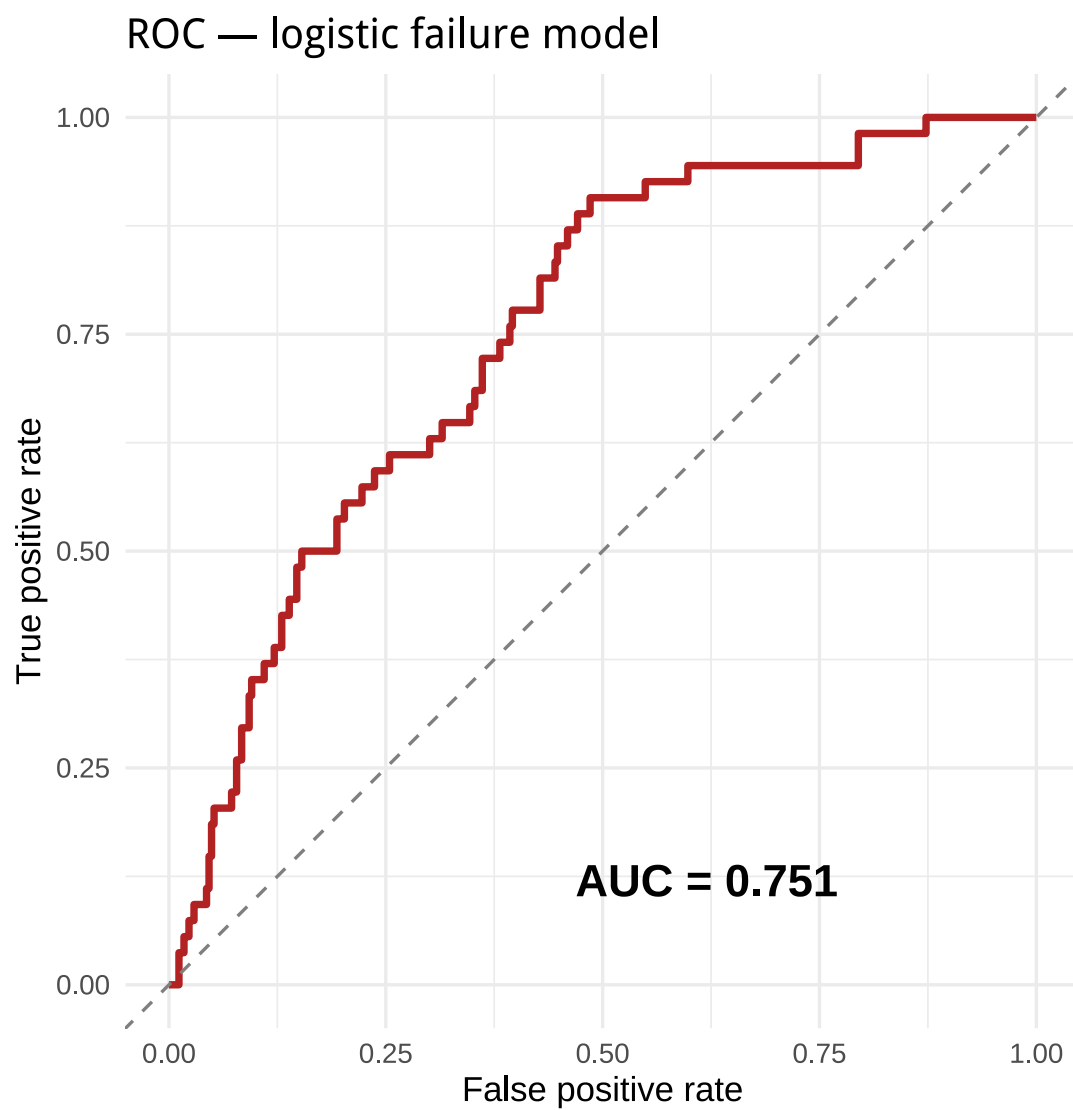


Figure 9: ROC curve for the pipeline failure model.

```
mutate(
  IRI = 0.8 + 0.055 * age + 0.18 * traffic + 0.08 * rut + rnorm(n_sec, 0, 0.45),
  condition = cut(IRI,
    breaks = c(-Inf, 2.7, 4.2, Inf),
    labels = c("Good", "Fair", "Poor")
  )
)
table(pavement$condition)

##
## Good Fair Poor
## 117 207 76
```

What you are seeing: The table shows how many pavement sections fall into each condition class (Good, Fair, Poor). The classes are ordered by IRI thresholds used in highway practice. A well-balanced dataset has all three classes represented; if one class dominates, we would worry about class imbalance later.

4.1.3 Train / test split and tree fitting

```
set.seed(7)
idx <- sample(seq_len(n_sec), size = 0.7 * n_sec)
train_p <- pavement[idx, ]
test_p <- pavement[-idx, ]

tree_pav <- rpart(condition ~ age + traffic + rut,
  data = train_p,
  method = "class",
  control = rpart.control(minsplit = 20, cp = 0.01)
)

rpart.plot(tree_pav,
  type = 4, extra = 104, box.palette = "auto",
  shadow.col = "grey", cex = 0.85
)
```

How to read the tree diagram: Start at the top node — the tree has chosen the single most informative question to ask first. Follow the yes/no branches downward. Each **leaf** (bottom box) shows the majority class and the class probabilities. Here the tree typically asks about **traffic** or **age** first — rediscovering that heavy, old pavements deteriorate first. Colour depth indicates the predicted class.

4.1.4 How a tree carves up feature space

```
tree_2d <- rpart(condition ~ age + traffic,
  data = train_p, method = "class",
  control = rpart.control(cp = 0.01)
)
grid_p <- expand.grid(
  age = seq(1, 25, length.out = 200),
```

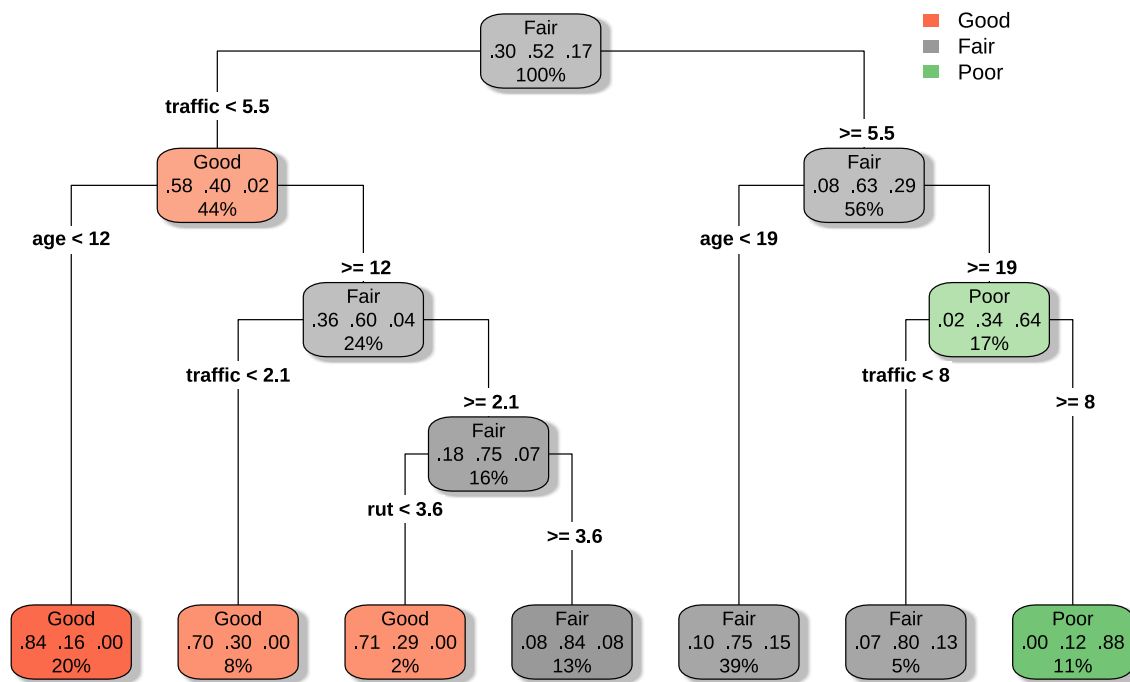


Figure 10: The learned decision tree. Each box shows the majority class and class probabilities.

```

traffic = seq(0.2, 12, length.out = 200)
)
grid_p$pred <- predict(tree_2d, grid_p, type = "class")

ggplot() +
  geom_raster(data = grid_p, aes(age, traffic, fill = pred), alpha = 0.45) +
  geom_point(data = train_p, aes(age, traffic, colour = condition), size = 1.2) +
  scale_fill_manual(
    values = c(Good = "palegreen3", Fair = "khaki1", Poor = "indianred1"),
    name = "Predicted"
  ) +
  scale_colour_manual(
    values = c(Good = "darkgreen", Fair = "darkgoldenrod3", Poor = "firebrick"),
    name = "Actual"
  ) +
  labs(
    title = "Tree partitions of the age x traffic plane",
    x = "Pavement age (years)", y = "Traffic (M ESALs/yr)"
  )

```

How to read this figure: The background colour shows what the tree *predicts* at each (age, traffic) coordinate. The dots are the actual data. Notice the **rectangular, axis-aligned boundaries** — a tree splits one feature at a time. That is both a strength (easy to interpret) and a weakness (boundaries look blocky compared to reality).

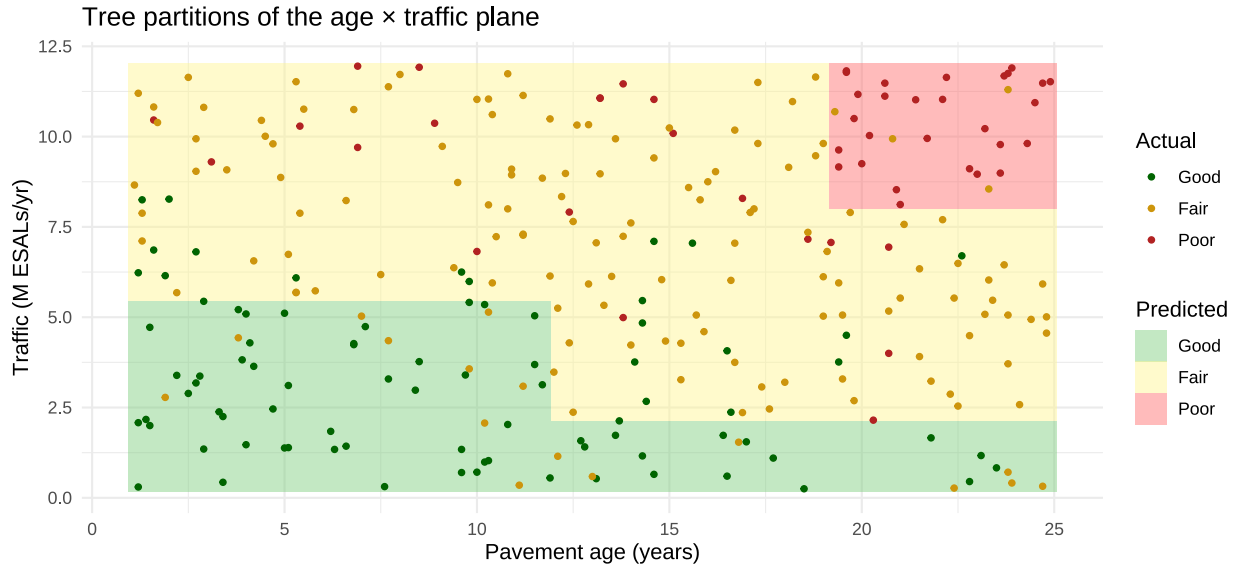


Figure 11: Decision trees create rectangular regions — axis-aligned cuts, one at a time.

4.1.5 Pruning: choosing tree size with the complexity parameter

```
plotcp(tree_pav)
```

How to read this plot: The horizontal axis is the complexity parameter `cp` (larger `cp` = simpler tree). The vertical axis is cross-validated error. The dashed horizontal line is one standard error above the minimum. **The rule:** pick the simplest tree whose error is still within that band — this avoids overfitting while keeping the model small enough to explain to a maintenance crew.

4.1.6 Test performance

```
pred_tree <- predict(tree_pav, newdata = test_p, type = "class")
cm_tree <- table(Predicted = pred_tree, Actual = test_p$condition)
cm_tree
```

```
##           Actual
## Predicted Good Fair Poor
##      Good   26   9   0
##      Fair    7  48  17
##      Poor    0   3  10
```

```
sprintf("Test accuracy = %.1f%%", 100 * sum(diag(cm_tree)) / sum(cm_tree))
```

```
## [1] "Test accuracy = 70.0%"
```

How to read the confusion matrix: Rows are what the tree predicted, columns are what actually happened. The diagonal cells (top-left, middle, bottom-right) are correct classifications. Off-diagonal cells are mistakes — for a pavement manager, a “Poor predicted as Good” mistake is the most dangerous (it leads to no intervention on a road that needs one).

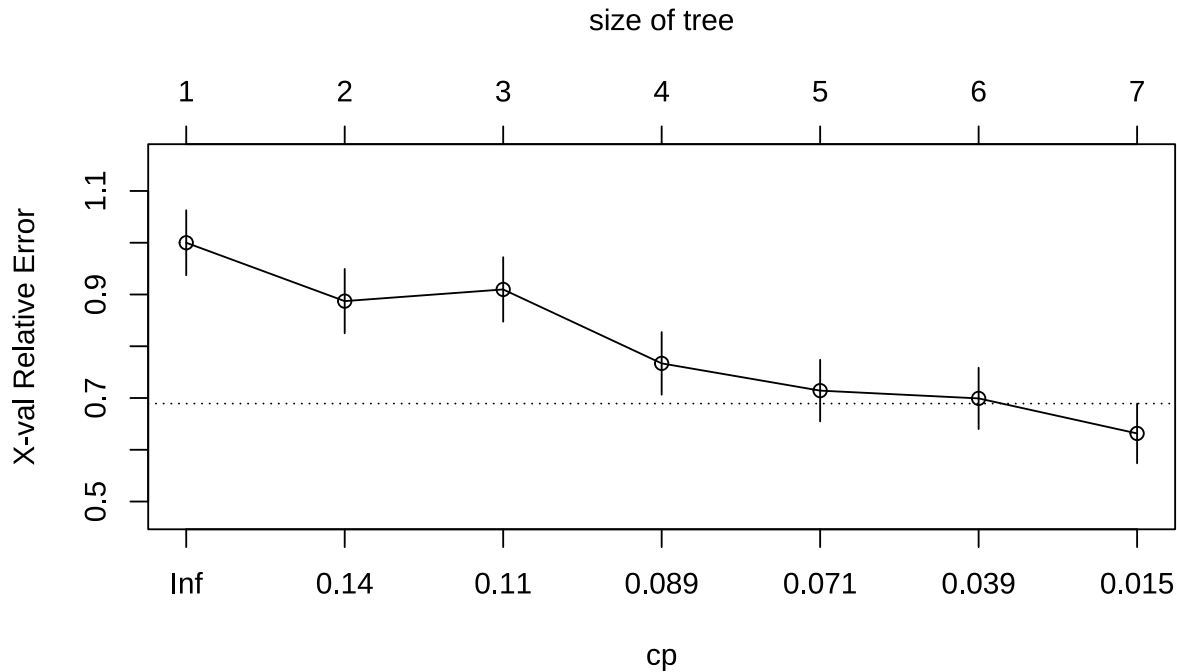


Figure 12: `plotcp()`: pick the smallest tree within one SE of the minimum — the same overfitting story as Unit 3.

4.2 Random Forests

4.2.1 The idea in one line

Train many decorrelated trees on **bootstrap samples** with **random feature subsets**, then vote (classification) or average (regression). The wisdom of the crowd beats any single expert.

$$\hat{y} = \text{majority}\{T_1(x), T_2(x), \dots, T_B(x)\}$$

Civil analogy: One inspector can be biased; a **panel of five independent inspectors** averaging their condition ratings is far more reliable. Random forest = a panel of 500 slightly-different decision trees. Bonus: each tree never saw ~37% of the data (**out-of-bag**, OOB) — a free built-in test set, like each inspector skipping some buildings and cross-checking the others.

```
rf_boxes <- tibble(
  x = rep(c(2.5, 6.0, 9.5), each = 3),
  y = c(3.0, 2.2, 3.0, 2.2, 3.0, 2.2, 3.0, 2.2, 3.0),
  lab = "Tree",
  fill = "#4C72B0"
)
ggplot() +
  annotate("rect", xmin = 1, xmax = 4, ymin = 0.8, ymax = 4.2, fill = "grey95", colour = "grey60") +
  annotate("text", x = 2.5, y = 4.5, label = "Bootstrap samples\n+ random features", size = 3.2) +
  geom_point(data = rf_boxes, aes(x, y), shape = 15, size = 9, colour = "#4C72B0") +
  annotate("text", x = 6.0, y = 2.6, label = "500\ntrees\nvote", size = 3.4, fontface = "bold") +
  annotate("rect", xmin = 8.2, xmax = 10.8, ymin = 1.8, ymax = 3.4, fill = "#C44E52", colour = "grey20")
```

```

annotate("text", x = 9.5, y = 2.6, label = "Majority vote\n= prediction", colour = "white", fontface = "bold")
annotate("segment",
  x = 4.05, xend = 5.3, y = 2.6, yend = 2.6,
  arrow = arrow(length = unit(0.25, "cm")), linewidth = 1
) +
annotate("segment",
  x = 6.7, xend = 8.15, y = 2.6, yend = 2.6,
  arrow = arrow(length = unit(0.25, "cm")), linewidth = 1
) +
xlim(0.5, 11) +
ylim(0.5, 5) +
theme_void() +
labs(title = "Random forest = bagged, decorrelated decision trees")

```

Random forest = bagged, decorrelated decision trees

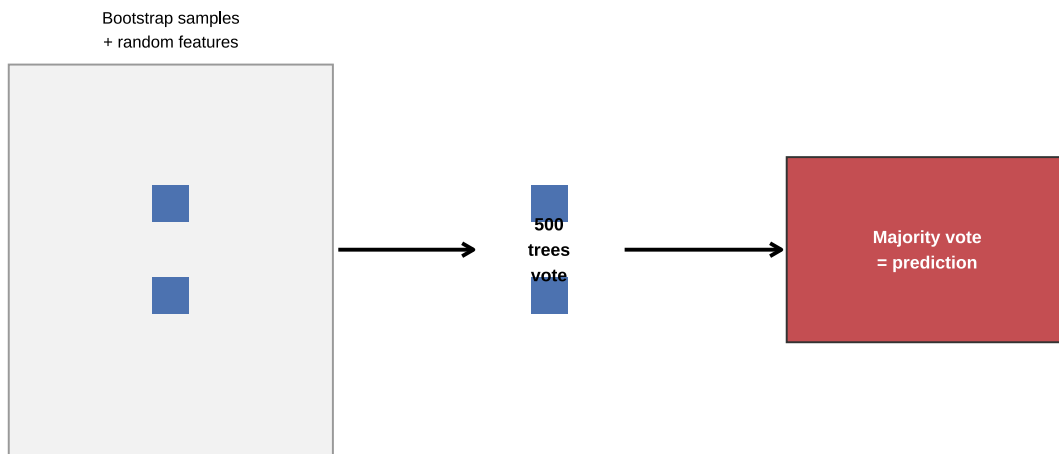


Figure 13: Bagging: bootstrap samples → diverse trees → majority vote.

How to read this schematic: Each tree is trained on a different bootstrap sample (random rows with replacement) and only considers a random subset of features at each split. This “forced diversity” is what makes the trees disagree — and disagreement is what makes the crowd smarter than any one member. The final prediction is a majority vote (or, in regression, an average).

4.2.2 Simulate a flood susceptibility dataset

Flood susceptibility mapping uses **terrain + climate factors**: low elevation, gentle slope, high rainfall, close to river, sparse vegetation (low NDVI) → flood-prone.

```

set.seed(41)
n_sites <- 600
flood_sites <- tibble(
  elevation = round(runif(n_sites, 5, 200), 1), # metres
  slope     = round(runif(n_sites, 0.5, 25), 1), # percent
  rainfall  = round(runif(n_sites, 600, 1800), 0), # mm/year

```

```

dist_river = round(runif(n_sites, 20, 1500), 0), # metres
ndvi       = round(runif(n_sites, 0.05, 0.8), 2)
) %>%
mutate(
  logit = 1.6 - 0.035 * elevation - 0.09 * slope + 0.0035 * rainfall -
    0.0022 * dist_river - 2.0 * ndvi,
  flood = factor(ifelse(rbinom(n_sites, 1, plogis(logit)) == 1, "yes", "no"),
    levels = c("no", "yes")
  )
)
mean(flood_sites$flood == "yes") # prevalence of flood observations

```

```
## [1] 0.325
```

What you are seeing: The last number is the **base rate of flooding** in the simulated dataset. It typically falls in the 10–30% range — a moderately imbalanced problem. That is realistic for susceptibility studies: floods happen, but rarely at any given site.

4.2.3 Train the forest

```

set.seed(42)
f_idx <- sample(seq_len(n_sites), 0.7 * n_sites)
flood_train <- flood_sites[f_idx, ]
flood_test <- flood_sites[-f_idx, ]

rf_flood <- randomForest(flood ~ elevation + slope + rainfall + dist_river + ndvi,
  data = flood_train, ntree = 500, importance = TRUE
)
rf_flood

```

```
##
```

```
## Call:
```

```
## randomForest(formula = flood ~ elevation + slope + rainfall + dist_river + ndvi, data = flood_train)
```

```
## Type of random forest: classification
```

```
## Number of trees: 500
```

```
## No. of variables tried at each split: 2
```

```
##
```

```
## OOB estimate of error rate: 15.95%
```

```
## Confusion matrix:
```

```
## no yes class.error
```

```
## no 263 25 0.08680556
```

```
## yes 42 90 0.31818182
```

How to read the printed forest: The output prints the number of trees, the OOB (out-of-bag) confusion matrix, and the OOB error rate for each class. **The OOB error is a free validation estimate** — it was computed on data each tree never saw, so it is honest. If OOB error is ~0.1, the forest is achieving ~90% accuracy on held-out rows.

4.2.4 Watch the OOB error converge as trees are added

```
as.data.frame(rf_flood$err.rate) %>%
  mutate(tree = row_number()) %>%
  pivot_longer(-tree, names_to = "metric") %>%
  ggplot(aes(tree, value, colour = metric)) +
  geom_line(linewidth = 0.8) +
  labs(
    title = "Random forest learning curve (OOB error)",
    x = "Number of trees", y = "Error rate", colour = NULL
  )
```

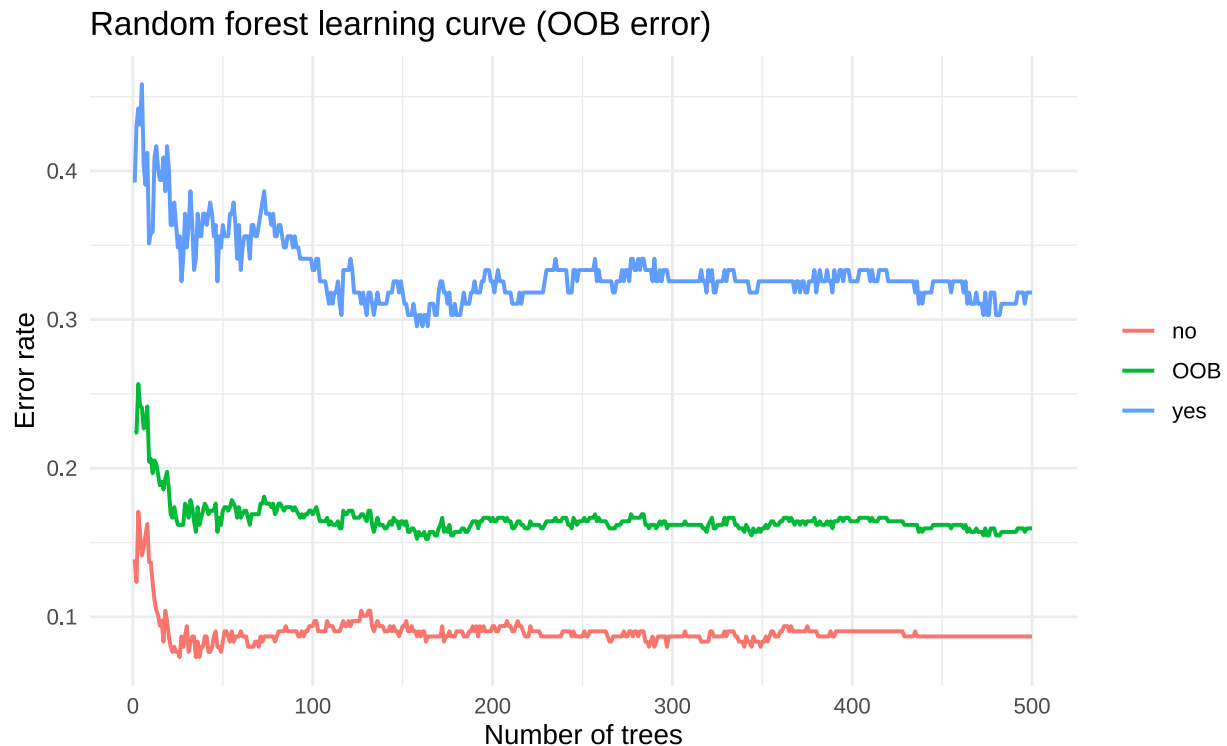


Figure 14: Out-of-bag error vs number of trees. After 200 trees, adding more helps very little — that is the forest 'converging'.

How to read this figure: Each curve tracks the OOB error for one class (and the overall). Error drops steeply at first, then flattens. The **knee** (usually around 100–250 trees) is where adding more trees stops helping. Beyond that you are just burning CPU time. This is how practitioners decide **ntree**.

4.2.5 Which factors drive flooding? Variable importance

```
varImpPlot(rf_flood, main = "Flood susceptibility: which factors matter most?")
```

How to read the variable importance plot: Two panels are printed — one for mean decrease in accuracy, one for mean decrease in Gini. Higher = more important. **rainfall** and **dist_river** typically dominate

Flood susceptibility: which factors matter most?

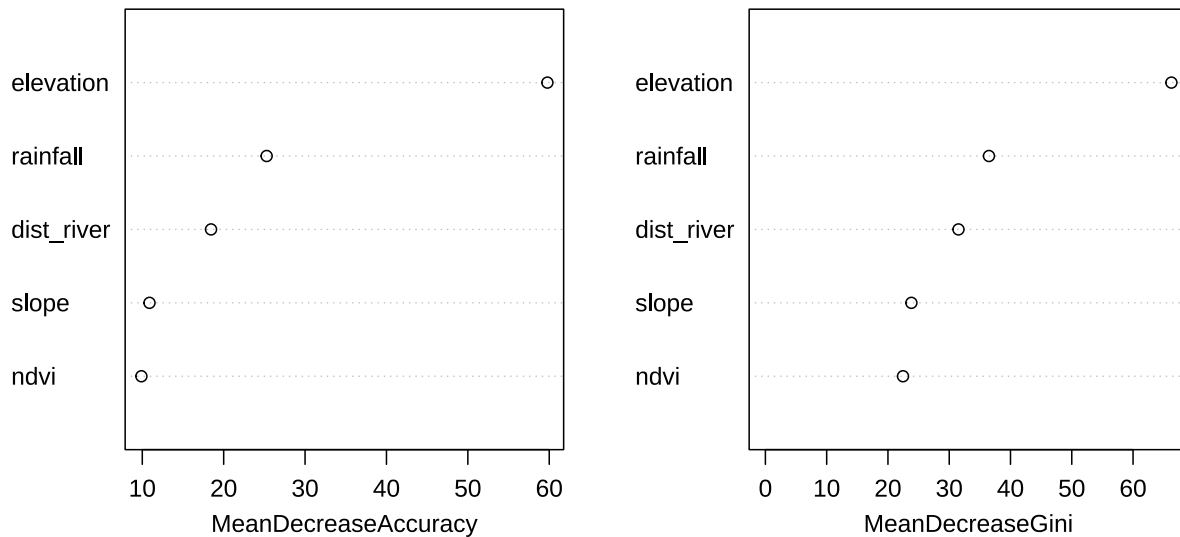


Figure 15: Variable importance: rainfall and distance-to-river dominate — matches hydrological intuition.

— matches hydrological intuition (more rain + closer to a river = more flooding). **elevation** matters next; **ndvi** is usually weakest in this setup. If a factor you *know* matters shows up near the bottom, investigate — that is a signal your features or simulation may need rethinking.

4.2.6 Partial dependence: how does probability respond to one factor?

```
pd_rain <- partial(rf_flood,
  pred.var = "rainfall",
  which.class = "yes", train = flood_train,
  prob = TRUE, plot = FALSE
)
pd_elev <- partial(rf_flood,
  pred.var = "elevation",
  which.class = "yes", train = flood_train,
  prob = TRUE, plot = FALSE
)

p1 <- ggplot(pd_rain, aes(rainfall, yhat)) +
  geom_line(colour = "steelblue", linewidth = 1.2) +
  labs(
    title = "Effect of rainfall",
    x = "Annual rainfall (mm)", y = "P(flood)"
  )

p2 <- ggplot(pd_elev, aes(elevation, yhat)) +
  geom_line(colour = "firebrick", linewidth = 1.2) +
```

```
labs(
  title = "Effect of elevation",
  x = "Elevation (m)", y = "P(flood)"
)
p1 | p2
```

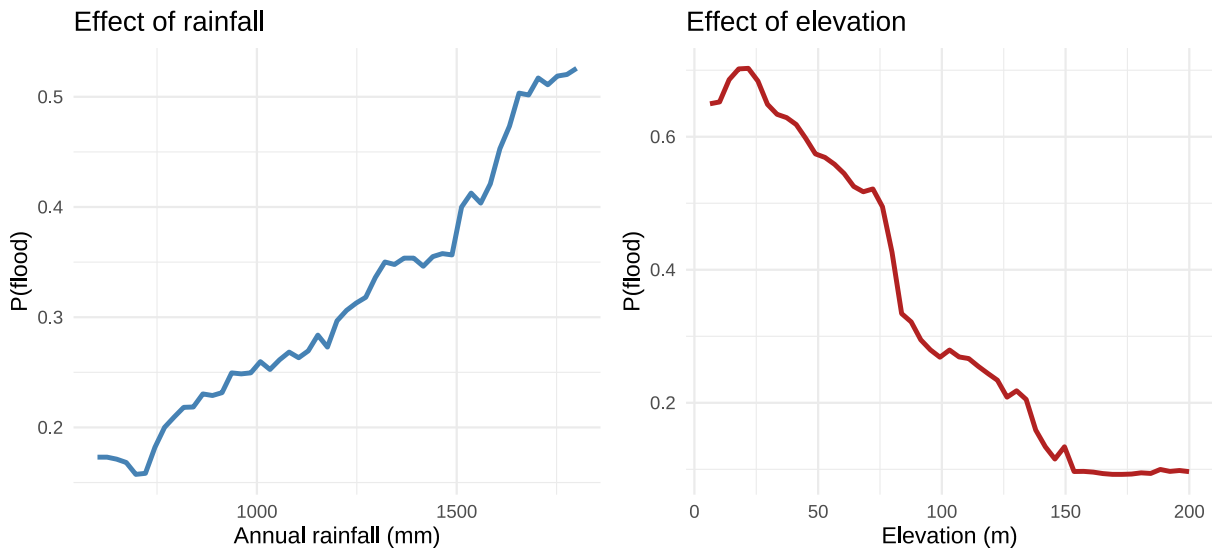


Figure 16: Partial dependence: holding everything else constant, flood probability rises with rainfall and falls with elevation.

How to read partial dependence plots: These show the model's *average* predicted probability as one feature sweeps across its range, holding everything else constant. **Steep curves = strong effect;** flat curves = the model does not use that feature much. A monotonic rise with rainfall and monotonic fall with elevation is physically sensible — a curved or non-monotonic shape would reveal a **threshold** the forest has discovered on its own.

4.2.7 Test-set ROC

```
prob_rf <- predict(rf_flood, newdata = flood_test, type = "prob")[, "yes"]
roc_rf <- roc(flood_test$flood, prob_rf,
  levels = c("no", "yes"),
  direction = "<", quiet = TRUE
)
sprintf("Test AUC = %.3f", auc(roc_rf))
```

```
## [1] "Test AUC = 0.908"
```

How to read the AUC: A number is printed — the area under the ROC on the held-out test sites. **AUC = 0.5** is guessing; **AUC = 0.9+** is excellent. Values above 0.85 are typical for well-featured flood susceptibility models. This is the honest number to quote in a report — not the OOB error, and never the training error.

4.2.8 Bonus application: crop yield (regression forest)

```
set.seed(51)
n_farm <- 350
farms <- tibble(
  rainfall = round(runif(n_farm, 400, 1600), 0), # mm / season
  fertilizer = round(runif(n_farm, 40, 180), 0), # kg/ha
  soil_index = round(runif(n_farm, 1, 10), 1), # 1 (poor) to 10 (excellent)
  ndvi_peak = round(runif(n_farm, 0.2, 0.85), 2)
) %>%
  mutate(yield = round(pmax(1.2 + 0.004 * rainfall + 0.012 * fertilizer +
    0.25 * soil_index + 2.2 * ndvi_peak +
    rnorm(n_farm, 0, 0.5), 0.5), 2))

rf_yield <- randomForest(yield ~ ., data = farms, ntree = 400, importance = TRUE)
sprintf("OOB R2 = %.3f", tail(rf_yield$rsq, 1))
```

```
## [1] "OOB R2 = 0.819"
```

What you are seeing: The last number is the **out-of-bag R²** for the yield regression. Above 0.7 = the model explains most yield variation. Random forests work for regression exactly as they do for classification: averaging individual tree predictions gives a smoother, more accurate result than any single tree.

```
imp_df <- as_tibble(importance(rf_yield)[, "%IncMSE"], rownames = "variable")
colnames(imp_df) <- c("variable", "value")

p_imp <- ggplot(imp_df, aes(x = reorder(variable, value), y = value)) +
  geom_col(fill = "darkgreen") +
  coord_flip() +
  labs(title = "Yield drivers (%IncMSE)", x = NULL, y = "% increase in MSE")

p_scatter <- tibble(observed = farms$yield, oob = rf_yield$predicted) %>%
  ggplot(aes(oob, observed)) +
  geom_point(alpha = 0.55, colour = "darkgreen") +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed") +
  labs(
    title = "OOB predictions vs observations",
    x = "Predicted yield (t/ha)", y = "Observed yield (t/ha)"
  ) +
  coord_equal()

p_imp | p_scatter
```

How to read the two panels:

- **Left:** Variable importance. Bars are the % increase in MSE that results when each variable is shuffled (larger = more important). The top bar is the yield driver the forest relies on most — typically `ndvi_peak` or `rainfall` in this simulation.
- **Right:** OOB predicted vs observed yield. Points near the dashed 1:1 line = good predictions. A wide scatter cloud or systematic curvature means the forest is struggling with some part of the range.

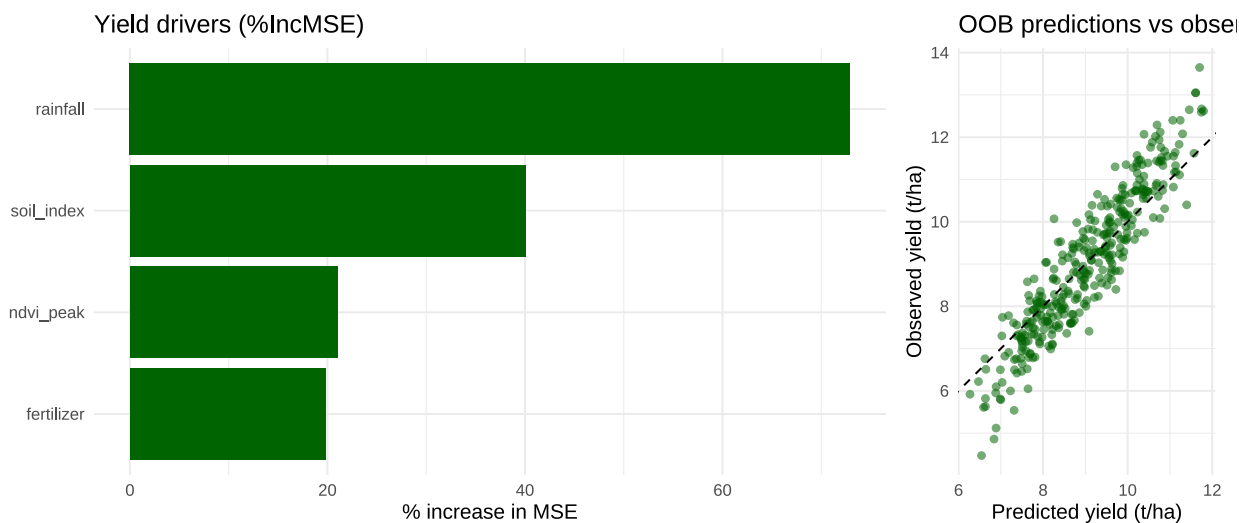


Figure 17: Crop yield forest: variable importance (left) and OOB-predicted vs observed yield (right).

Why forests are the workhorse of geospatial ML: They handle mixed units (m, %, mm, index), need no scaling, capture non-linear thresholds (floods are threshold phenomena), and give importance rankings that map directly onto factor maps used in susceptibility analysis.

5 Proximity and Probability Models

5.1 K-Nearest Neighbors (KNN)

5.1.1 The idea in one line

To classify a new point, look at the k closest training points and take a majority vote. Distance (Euclidean):

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots}$$

Civil analogy: You send a soil sample to the lab and the lab tech says: “*It looks, feels and behaves like the samples from the three nearest boreholes — sandy.*” No equation, just **similarity to known cases**. That is KNN. It is also exactly how engineers interpolate **groundwater levels between observation wells**.

5.1.2 Simulate a soil classification dataset

```
set.seed(61)
n_per <- 80
soil <- rbind(
  tibble(sand = rnorm(n_per, 72, 8), clay = rnorm(n_per, 8, 4), soil_type = "Sandy"),
  tibble(sand = rnorm(n_per, 45, 10), clay = rnorm(n_per, 25, 6), soil_type = "Loamy"),
  tibble(sand = rnorm(n_per, 20, 8), clay = rnorm(n_per, 55, 9), soil_type = "Clayey")
) %>%
```

```
mutate(
  sand = pmin(pmax(round(sand), 0), 100),
  clay = pmin(pmax(round(clay), 0), 100)
)
```

What you are seeing: A 240-row dataset with three soil classes, each described by sand% and clay%. Sandy soils have high sand, clayey soils have high clay, loamy sits in the middle. The `pmin/pmax` clipping keeps unrealistic percentages inside 0–100.

5.1.3 See the algorithm “think”: neighborhoods of a new sample

```
new_soil <- tibble(sand = 48, clay = 18)
d_new <- sqrt((soil$sand - new_soil$sand)^2 + (soil$clay - new_soil$clay)^2)
ord <- order(d_new)
k3 <- soil[ord[1:3], ]
k7 <- soil[ord[1:7], ]

circle <- function(cx, cy, r, n = 240) {
  t <- seq(0, 2 * pi, length.out = n)
  tibble(x = cx + r * cos(t), y = cy + r * sin(t))
}

c3 <- circle(
  new_soil$sand, new_soil$clay,
  max(sqrt((k3$sand - new_soil$sand)^2 + (k3$clay - new_soil$clay)^2)) + 2
)

c7 <- circle(
  new_soil$sand, new_soil$clay,
  max(sqrt((k7$sand - new_soil$sand)^2 + (k7$clay - new_soil$clay)^2)) + 2
)

soil_cols <- c(Sandy = "goldenrod1", Loamy = "forestgreen", Clayey = "brown3")

ggplot() +
  geom_point(data = soil, aes(sand, clay, colour = soil_type), size = 1.8, alpha = 0.55) +
  geom_path(data = c7, aes(x, y), colour = "grey30", linetype = "dashed") +
  geom_path(data = c3, aes(x, y), colour = "grey10", linewidth = 0.6) +
  geom_point(
    data = k7, aes(sand, clay), shape = 21, fill = "white",
    colour = "black", size = 2.6
  ) +
  geom_point(data = new_soil, aes(sand, clay), shape = 17, size = 4.5) +
  annotate("text",
    x = 46, y = 4, size = 3.4,
    label = "New sample (triangle)\nk=3 votes: ?   k=7 votes: ?"
  ) +
  scale_colour_manual(values = soil_cols, name = "Soil class") +
  coord_equal() +
  labs(
    title = "K-nearest neighbors: classification by local vote",
    x = "Sand content (%)", y = "Clay content (%)"
  )
```

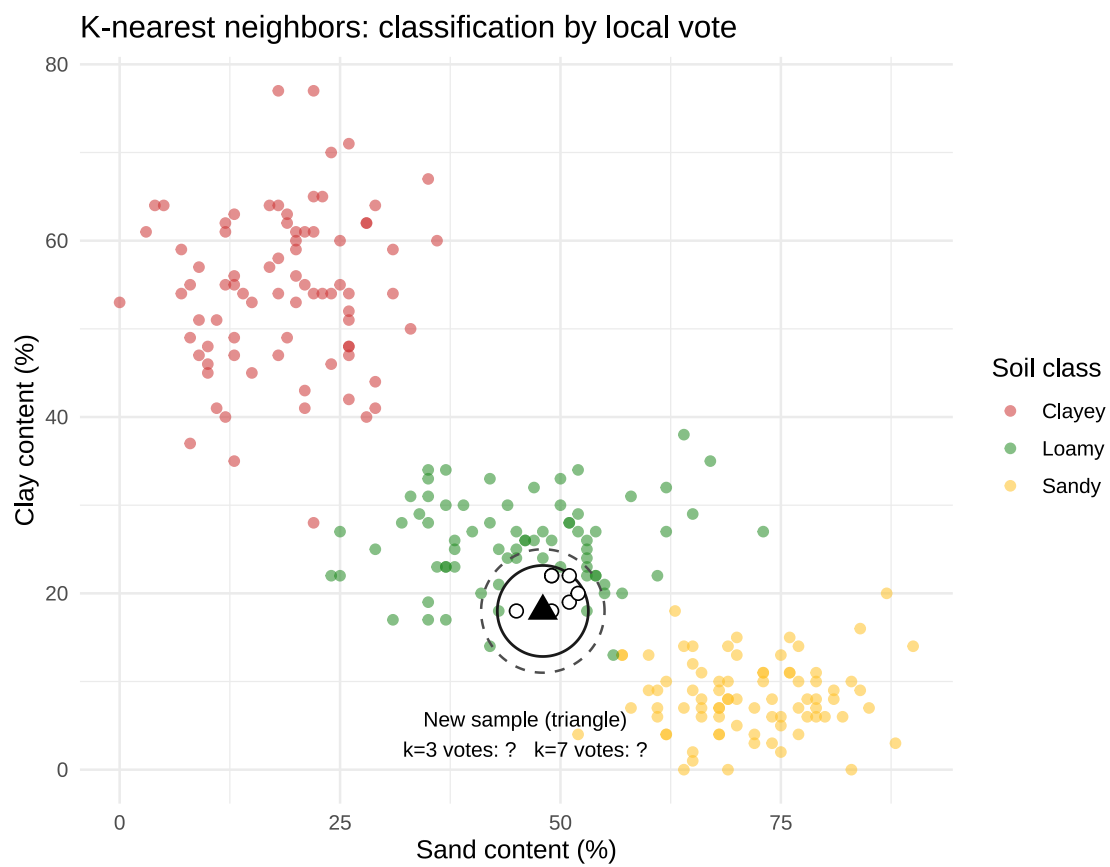


Figure 18: KNN in action: the dashed circle votes with $k = 7$ neighbors, the solid with $k = 3$.

How to read this figure: The black triangle is the new sample to classify. The two circles enclose its 3 and 7 nearest neighbours. $k = 3$ votes only with the closest 3 dots — a “local” opinion. $k = 7$ widens the circle, bringing in more neighbours — a “neighbourhood” opinion. Small k reacts to local noise; large k smooths out real structure. Section 6.4 shows how to choose k properly.

5.1.4 Decision boundaries: the effect of k

```
grid_s <- expand.grid(
  sand = seq(0, 100, length.out = 150),
  clay = seq(0, 100, length.out = 150)
)

make_knn_grid <- function(k) {
  g <- grid_s
  g$pred <- knn(
    train = soil[, c("sand", "clay")],
    test = g[, c("sand", "clay")],
    cl = soil$soil_type, k = k
  )
  g$k <- sprintf("k = %d", k)
  g
}

bind_rows(make_knn_grid(1), make_knn_grid(7), make_knn_grid(25)) %>%
  ggplot(aes(sand, clay, fill = pred)) +
  geom_raster(alpha = 0.4) +
  geom_point(
    data = soil,
    aes(x = sand, y = clay, colour = soil_type),
    inherit.aes = FALSE,
    size = 0.9, alpha = 0.7
  ) +
  scale_fill_manual(values = soil_cols, name = "Predicted") +
  scale_colour_manual(values = soil_cols, name = "Actual") +
  facet_wrap(~k) +
  coord_equal() +
  labs(
    title = "KNN decision boundaries for different k",
    x = "Sand (%)", y = "Clay (%)"
  )
}
```

How to read this figure: Three KNN boundaries are shown for $k = 1$, 7, and 25. $k = 1$ produces jagged, overfit boundaries that chase individual points — the bias-variance lesson from Unit 3 in action. $k = 25$ produces smooth but possibly over-generic boundaries. $k = 7$ is usually the sweet spot in problems this size.

5.1.5 Choosing k by validation

```
set.seed(62)
idx_s <- sample(seq_len(nrow(soil)), 0.7 * nrow(soil))
```

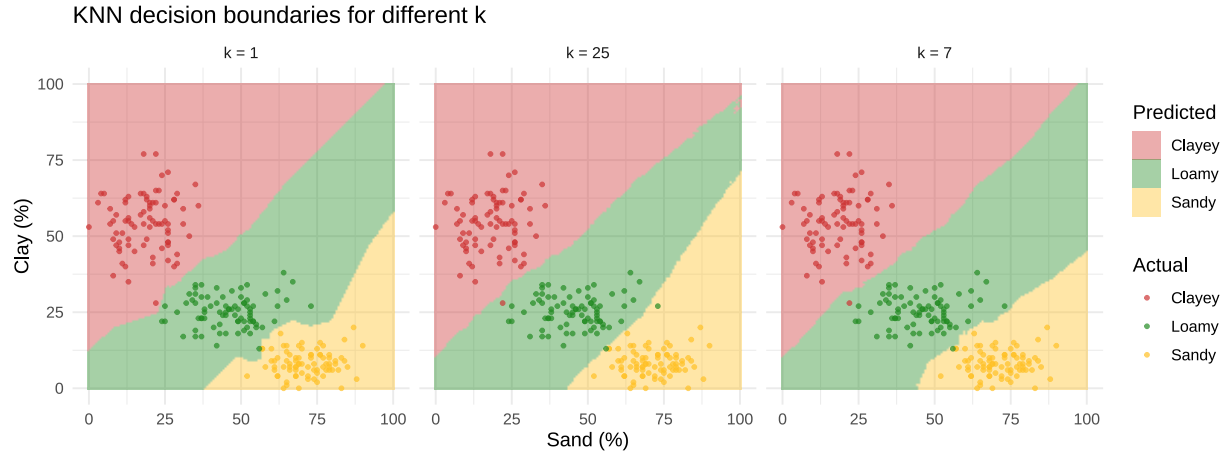


Figure 19: Small k = wiggly, overfit boundaries; large k = smooth, possibly underfit. Same bias-variance tradeoff from Unit 3.

```
soil_tr <- soil[idx_s, ]
soil_te <- soil[-idx_s, ]
ks <- seq(1, 31, by = 2)
acc_s <- sapply(ks, function(k) {
  p <- knn(
    train = soil_tr[, c("sand", "clay")],
    test = soil_te[, c("sand", "clay")],
    cl = soil_tr$soil_type, k = k
  )
  mean(p == soil_te$soil_type)
})
tibble(k = ks, accuracy = acc_s) %>%
  ggplot(aes(k, accuracy)) +
  geom_line(linewidth = 1, colour = "steelblue") +
  geom_point(colour = "firebrick", size = 2) +
  scale_x_continuous(breaks = ks) +
  labs(
    title = "Choosing k: test accuracy curve",
    x = "k (number of neighbors)", y = "Test accuracy"
  )
```

How to read this curve: The peak of the curve is the best k for *this dataset* — not a universal constant. Notice the drop at very small k (noise) and at very large k (over-smoothing). **Use odd k** for binary classification to avoid tied votes. Pick the k where the curve is both high *and* flat — a flat plateau means the choice is robust to small changes.

5.1.6 The scaling trap (very important!)

```
raw_demo <- data.frame(area_m2 = c(800, 5000), age_yrs = c(10, 12))
dist(raw_demo) # area dominates: 4200 vs 2 !!!
```

```
##      1
## 2 4200
```

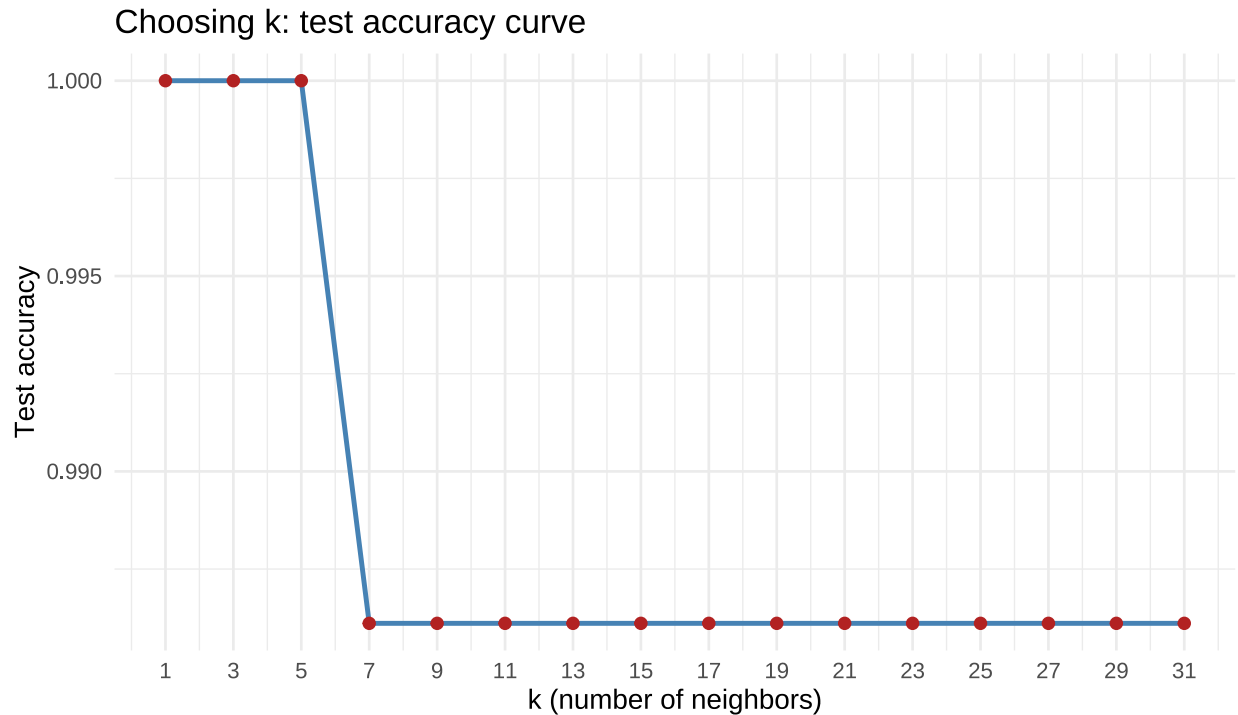


Figure 20: Accuracy vs k. Use odd k to avoid tied votes.

```
dist(scale(raw_demo)) # after z-standardisation both features count
```

```
## 1
## 2 2
```

How to read this output: The first distance is dominated by the `area_m2` difference (thousands) — the `age_yrs` difference (2) is invisible by comparison. The second distance, computed after z-standardising each column, gives both features equal weight. **This is why Unit 3’s scaling lesson matters so much for KNN.**

Rule: KNN uses raw distances, so **always scale features** (Unit 3 normalization!) when units differ — e.g., building area (m²) vs age (years), or rainfall (mm) vs slope (%).

5.2 Naive Bayes

5.2.1 The idea in one line

Use **Bayes’ theorem** with the “*naive*” assumption that features are independent within each class:

$$P(y \mid x_1, \dots, x_d) \propto P(y) \prod_{j=1}^d P(x_j \mid y)$$

Civil analogy: A bridge inspector decides “*corrosion distress likely?*” by combining the **prior** (bridges of this age & coastal exposure fail often) with each **symptom separately** (rust stains? spalling? drainage blocked?) without modeling interactions between symptoms. Crude but fast — and surprisingly strong as a screening tool.

```

ellipse_pts <- function(cx, cy, a, b, theta = 0, n = 240) {
  t <- seq(0, 2 * pi, length.out = n)
  tibble(
    x = cx + a * cos(t) * cos(theta) - b * sin(t) * sin(theta),
    y = cy + a * cos(t) * sin(theta) + b * sin(t) * cos(theta)
  )
}
truth <- ellipse_pts(85, 0.68, 26, 0.11, pi / 6)
naive <- ellipse_pts(85, 0.68, 26, 0.11, 0)
ggplot() +
  geom_path(
    data = truth, aes(x, y), colour = "firebrick",
    linewidth = 1.2, linetype = "solid"
  ) +
  geom_path(
    data = naive, aes(x, y), colour = "grey30",
    linewidth = 1.1, linetype = "dashed"
  ) +
  annotate("text",
    x = 85, y = 0.86, size = 3.4,
    label = "dashed = Naive Bayes assumption\nsolid = true correlated cloud"
  ) +
  coord_equal() +
  labs(
    title = "What 'naive' means: features treated as independent",
    x = "Rainfall (mm)", y = "Soil wetness (index)"
  )

```

How to read this figure: The solid red ellipse is the *true* joint distribution of rainfall and soil wetness in the flood class — they are correlated (wet ground + heavy rain come together). The dashed grey ellipse is the naive approximation — Naive Bayes pretends the axes are independent, drawing an axis-aligned cloud. It loses the tilt but gains computational speed and simplicity.

5.2.2 Simulate a flood-day dataset

```

set.seed(71)
nb_data <- rbind(
  tibble(rainfall = rnorm(350, 25, 12), soil_wetness = rnorm(350, 0.30, 0.12), flood_day = "no"),
  tibble(rainfall = rnorm(150, 85, 20), soil_wetness = rnorm(150, 0.70, 0.10), flood_day = "yes")
) %>%
  mutate(
    rainfall = round(pmax(rainfall, 0), 1),
    soil_wetness = round(pmin(pmax(soil_wetness, 0), 1), 2),
    flood_day = factor(flood_day, levels = c("no", "yes"))
  )

```

What you are seeing: 500 days split into two classes — 350 non-flood and 150 flood days. Flood days are characterised by higher rainfall and higher soil wetness. This is a deliberately **imbalanced** dataset (30% positives), which is realistic for flood events and exactly the case where a naive accuracy metric would mislead.

What 'naive' means: features treated as independent

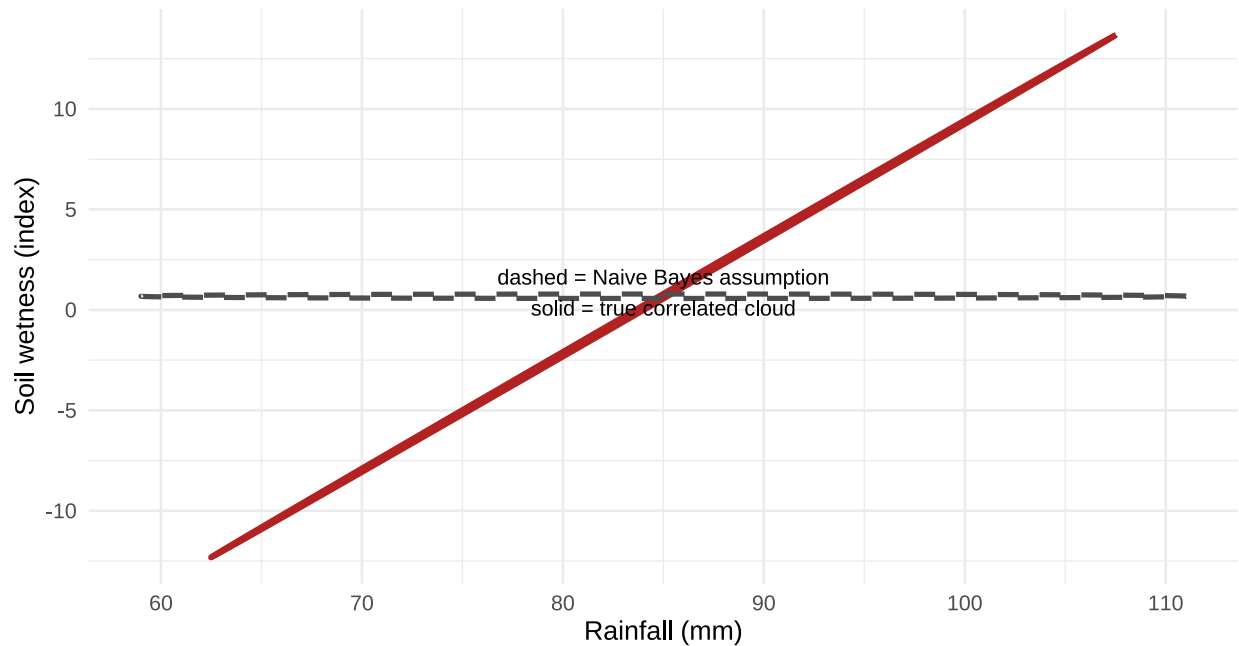


Figure 21: 'Naive' means we pretend the tilted (correlated) cloud is axis-aligned. Loss of realism, gain of speed.

```
p1 <- ggplot(nb_data, aes(rainfall, fill = flood_day)) +
  geom_density(alpha = 0.45) +
  labs(title = "Rainfall by flood day", x = "Rainfall (mm)")
p2 <- ggplot(nb_data, aes(soil_wetness, fill = flood_day)) +
  geom_density(alpha = 0.45) +
  labs(title = "Soil wetness by flood day", x = "Soil wetness index")
p1 | p2
```

How to read these density plots: Each panel shows the *class-conditional* distribution of one feature. The further apart the two coloured curves, the more useful that feature is to the classifier. Both curves are visibly separated here — rainfall more so than wetness — which predicts a well-performing Naive Bayes model. **Gaussian NB** summarises each curve with just a mean and a standard deviation.

5.2.3 Fit, inspect, predict

```
set.seed(72)
nb_idx <- sample(seq_len(nrow(nb_data)), 0.7 * nrow(nb_data))
nb_train <- nb_data[nb_idx, ]
nb_test <- nb_data[-nb_idx, ]

nb_fit <- naiveBayes(flood_day ~ rainfall + soil_wetness, data = nb_train)
nb_fit # prints the prior P(y) and per-class means/sds - the whole model!
```

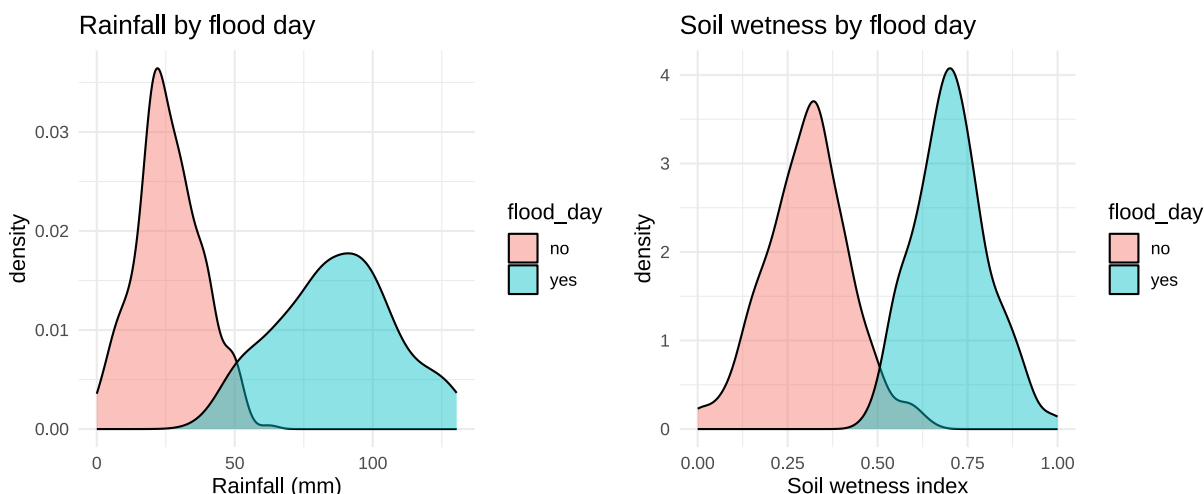


Figure 22: Class-conditional distributions: Naive Bayes summarizes each feature per class with a mean and sd (Gaussian NB).

```
##
## Naive Bayes Classifier for Discrete Predictors
##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace)
##
## A-priori probabilities:
## Y
##      no      yes
## 0.7114286 0.2885714
##
## Conditional probabilities:
##      rainfall
## Y      [,1]      [,2]
## no 26.73936 12.37045
## yes 86.28020 20.79552
##
##      soil_wetness
## Y      [,1]      [,2]
## no 0.2988353 0.1138647
## yes 0.7031683 0.1019503
```

How to read the printed model: The output is the *entire trained model* in a few lines. **A-priori probabilities** = the class priors (roughly 70/30). The two blocks below print the mean and standard deviation of each feature, split by class. That is all Naive Bayes stores — tiny, fast, interpretable. Contrast that with 500 decision trees or a deep neural network.

```
predict(nb_fit, newdata = tibble(rainfall = 60, soil_wetness = 0.55), type = "raw")
```

```
##              no      yes
## [1,] 0.05683548 0.9431645
```

How to read the posterior output: Two numbers are printed — the posterior probability of each class for a day with 60 mm rainfall and 0.55 wetness. The larger one wins. That transparency — showing both probabilities — is why NB is often used as a fast screening tool before a heavier model is deployed.

```
nb_grid <- expand_grid(
  rainfall = seq(0, 160, length.out = 180),
  soil_wetness = seq(0, 1, length.out = 180)
)
nb_grid$pred <- predict(nb_fit, newdata = nb_grid)

ggplot() +
  geom_raster(data = nb_grid, aes(rainfall, soil_wetness, fill = pred), alpha = 0.35) +
  geom_point(data = nb_test, aes(rainfall, soil_wetness, colour = flood_day), size = 1.7) +
  scale_fill_manual(values = c(no = "dodgerblue", yes = "firebrick"), name = "Predicted") +
  scale_colour_manual(values = c(no = "navy", yes = "red"), name = "Actual") +
  labs(
    title = "Naive Bayes: flood-day decision regions",
    x = "Rainfall (mm)", y = "Soil wetness index"
  )
)
```

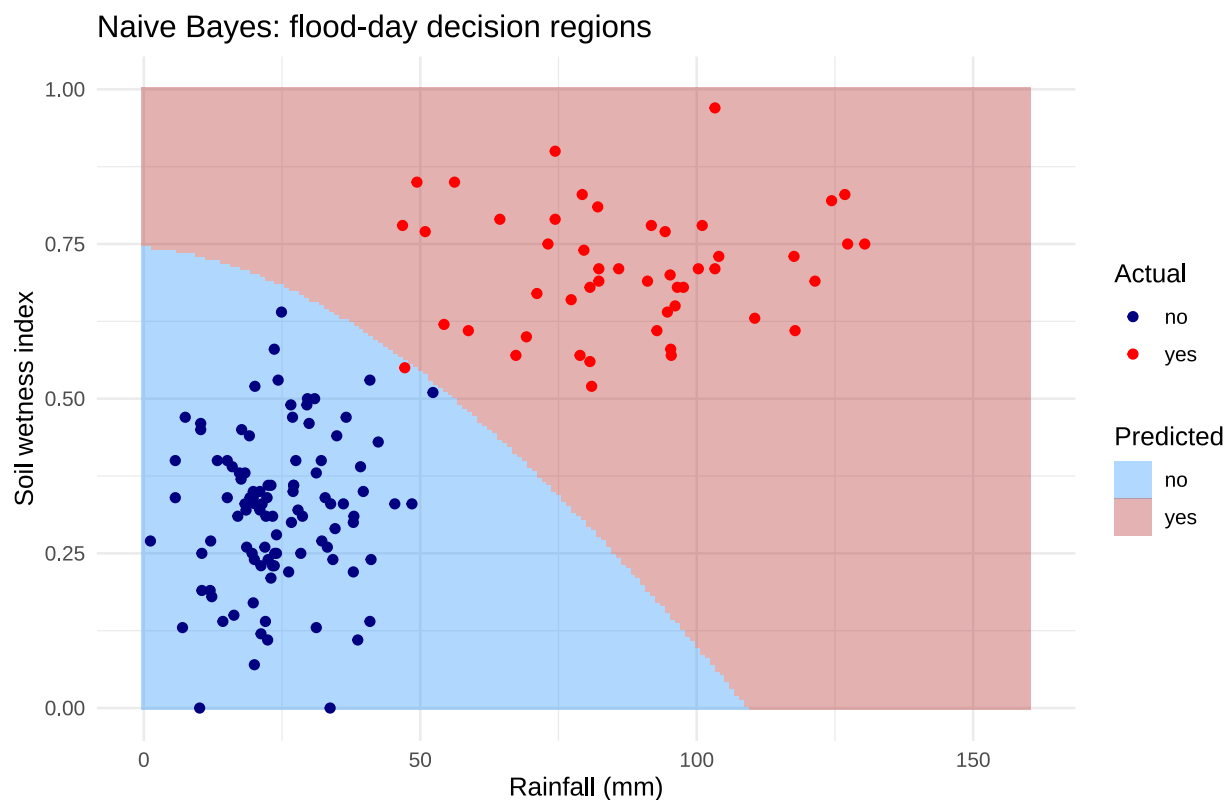


Figure 23: Naive Bayes decision regions with the held-out test points.

How to read this figure: The background colour is the predicted class at each point on the (rainfall, wetness) plane. The boundary between blue and red regions is the *decision surface* — a straight-ish line because NB is (essentially) a linear classifier in this feature space. Points are the held-out test set: blue = no flood, red = flood. The model is doing well when red points land in red regions.

```
pred_nb <- predict(nb_fit, newdata = nb_test)
sprintf("Naive Bayes test accuracy = %.1f%%", 100 * mean(pred_nb == nb_test$flood_day))
```

```
## [1] "Naive Bayes test accuracy = 99.3%"
```

How to read this number: This is the *test accuracy* — how often the model got the right class on data it never saw. Given how separated the two distributions are, accuracy above 90% is expected. **But remember Unit 3’s warning:** for rare, dangerous events like floods, always check recall next to accuracy.

6 Clustering Techniques for Regional Analysis

Why clustering for a civil engineer? Planning does not start with labeled data — it starts with “*give me homogeneous zones.*” Water-supply design zones, transport fare zones, maintenance contract packages, landslide-prone vs stable regions: all are **unsupervised grouping** problems. Here the labels do not exist; the algorithm invents them.

6.1 K-Means

6.1.1 The idea in one line

Choose K centroids, assign each point to the nearest centroid, recompute centroids, repeat — minimizing:

$$\sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

6.1.2 Simulate 360 settlements (urban / suburban / rural)

```
set.seed(81)
n_zone <- 120
regions <- rbind(
  tibble(pop_density = rnorm(n_zone, 8000, 1500), ndvi = rnorm(n_zone, 0.25, 0.06), zone = "Urban core"),
  tibble(pop_density = rnorm(n_zone, 2800, 700), ndvi = rnorm(n_zone, 0.45, 0.07), zone = "Suburban"),
  tibble(pop_density = rnorm(n_zone, 500, 180), ndvi = rnorm(n_zone, 0.65, 0.06), zone = "Rural")
) %>%
  mutate(
    pop_density = round(pmax(pop_density, 50)),
    ndvi = round(pmin(pmax(ndvi, 0.02), 0.95), 3)
  )
```

What you are seeing: 360 simulated settlements, each described by population density and NDVI (greenness). Three true zones exist in the simulation — dense urban cores with low NDVI, mid-density suburbs, and sparse rural areas with high NDVI. **The clustering algorithm will not see these labels** — its job is to rediscover them from the raw coordinates alone.

6.1.3 Choosing K: elbow method + silhouette

```
X <- regions %>%
  select(pop_density, ndvi) %>%
  scale()
ks <- 1:10
wss <- sapply(ks, function(k) kmeans(X, centers = k, nstart = 25)$tot.withinss)
avg_sil <- sapply(ks[-1], function(k) {
  km <- kmeans(X, centers = k, nstart = 25)
  mean(silhouette(km$cluster, dist(X))[, 3])
})

p_elbow <- tibble(k = ks, wss = wss) %>%
  ggplot(aes(k, wss)) +
  geom_line(linewidth = 1, colour = "steelblue") +
  geom_point(size = 2) +
  geom_vline(xintercept = 3, linetype = "dashed", colour = "firebrick") +
  labs(title = "Elbow method", x = "K", y = "Total within-cluster SS")

p_sil <- tibble(k = ks[-1], sil = avg_sil) %>%
  ggplot(aes(k, sil)) +
  geom_line(linewidth = 1, colour = "darkgreen") +
  geom_point(size = 2) +
  geom_vline(xintercept = 3, linetype = "dashed", colour = "firebrick") +
  labs(title = "Average silhouette width", x = "K", y = "Mean silhouette")

p_elbow | p_sil
```

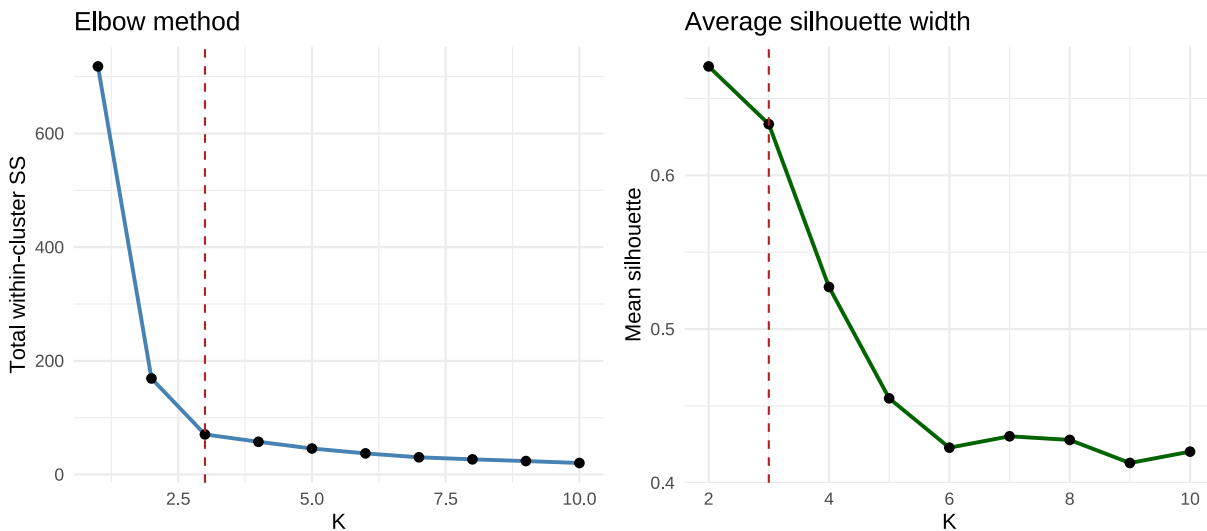


Figure 24: Elbow (left) — read it like a load-deflection curve: the 'yield point' is $K = 3$. Silhouette (right) agrees.

How to read these two plots:

- **Elbow (left):** WSS (total within-cluster sum of squares) always decreases as K grows — the trick is

to find the *knee* where improvement suddenly slows. That knee is usually the right K. Here it sits near $K = 3$.

- **Silhouette (right):** A measure of how well each point fits its own cluster (range -1 to $+1$). Higher is better. The peak usually agrees with the elbow — a reassuring cross-check.

Never trust one method alone. When both agree, the choice of K is on solid ground.

6.1.4 Final clustering and interpretation

```
km3 <- kmeans(X, centers = 3, nstart = 25)
regions$cluster <- factor(km3$cluster)

centroids <- sweep(
  sweep(km3$centers, 2, attr(X, "scaled:scale"), `*`),
  2, attr(X, "scaled:center"), `+`
) %>%
  as_tibble() %>%
  mutate(cluster = factor(1:3))

ggplot(regions, aes(pop_density, ndvi, colour = cluster)) +
  geom_point(size = 2, alpha = 0.7) +
  geom_point(
    data = centroids, aes(pop_density, ndvi, colour = cluster),
    shape = 18, size = 6
  ) +
  scale_colour_viridis_d(name = "Cluster") +
  labs(
    title = "K-means regional zoning (diamonds = centroids)",
    x = "Population density (per km²)", y = "NDVI (green cover)"
  ) +
  coord_equal()
```

How to read this figure: Each point is a settlement, coloured by the cluster the algorithm assigned. Diamonds are the three centroids — the “representative” point of each cluster. Notice how three distinct bands emerge, matching the true urban/suburban/rural split — even though the algorithm never saw the true labels. **The algorithm’s job stops here; your job is to *name* the clusters.**

```
regions %>%
  group_by(cluster) %>%
  summarise(
    n = n(),
    mean_pop = round(mean(pop_density)),
    mean_ndvi = round(mean(ndvi), 2)
  ) %>%
  arrange(desc(mean_pop))
```

```
## # A tibble: 3 x 4
##   cluster      n mean_pop mean_ndvi
##   <fct>   <int>   <dbl>   <dbl>
## 1 3         119    8204     0.25
## 2 1         114    2773     0.45
## 3 2         127     668     0.64
```

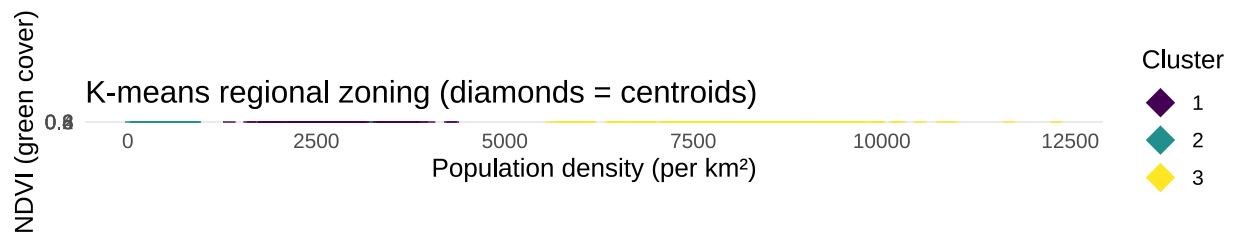


Figure 25: K-means zones ($K = 3$) with centroids. The algorithm never saw the true labels!

How to read this summary: For each cluster the table shows how many settlements, the mean population density, and the mean NDVI. **The cluster with the highest mean_pop and lowest mean_ndvi is your urban core; the one with the lowest density and highest greenness is rural.** Naming clusters is human work — the algorithm only gives numbers.

```
table(Cluster = regions$cluster, True_zone = regions$zone)
```

```
##           True_zone
## Cluster Rural Suburban Urban core
##      1      2      111      1
##      2     118      9      0
##      3      0      0     119
```

How to read this cross-tab: Rows are K-means clusters, columns are the *true* labels used in the simulation. A perfect clustering would put all counts on a single diagonal — one cluster per true zone. Real clustering usually has a few misassignments at the boundaries between zones, which is normal.

6.2 Hierarchical Clustering

6.2.1 The idea in one line

Start with every point as its own cluster; repeatedly **merge the two closest clusters** (agglomerative). “Closest” depends on the **linkage** rule. The result is a **dendrogram** — cut it at a height to get K clusters.

Civil analogy: A dendrogram is like the **Work Breakdown Structure** of a megaproject: individual activities merge into work packages, packages into sections, sections into the project. Cutting the tree at any level gives you a different contract-packaging granularity.

```
set.seed(82)
h_idx <- sample(seq_len(nrow(regions)), 45)
h_data <- regions[h_idx, c("pop_density", "ndvi", "zone")]
h_scaled <- scale(h_data[, c("pop_density", "ndvi")])

hc_ward <- hclust(dist(h_scaled), method = "ward.D2")
plot(hc_ward,
     main = "Hierarchical clustering of settlements (Ward linkage)",
     sub = "", xlab = "", cex = 0.6, labels = FALSE
)
rect.hclust(hc_ward, k = 3, border = c("firebrick", "steelblue", "darkgreen"))

m <- length(hc_ward$height)
k_cut <- 3
h_cut <- mean(hc_ward$height[(m - k_cut + 1):(m - k_cut + 2)])
abline(h = h_cut, lty = 2, col = "red")
```

How to read the dendrogram: Each vertical leaf at the bottom is one settlement. The height at which two leaves join = the “distance” at which those clusters merged. **Short joining heights = very similar; tall joining heights = dissimilar.** The three coloured rectangles show a cut at K = 3, which produces three clusters. The dashed horizontal line is the cut height.

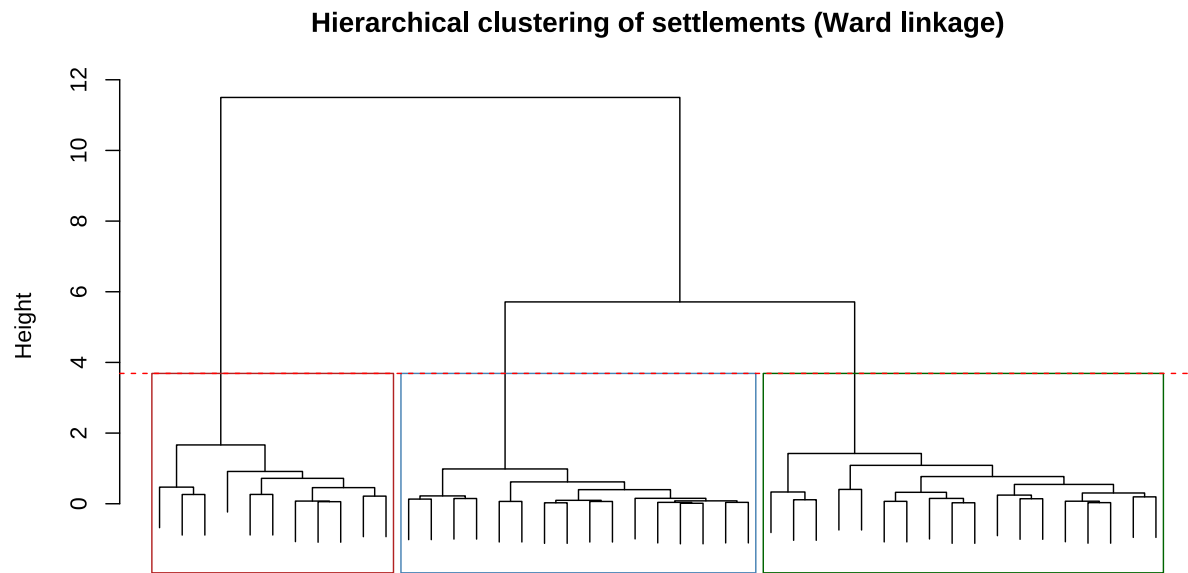


Figure 26: Ward dendrogram of 45 settlements; boxes show the $K = 3$ cut.

6.2.2 Linkage matters: same data, four dendrograms

```
methods <- c("single", "complete", "average", "ward.D2")
par(mfrow = c(2, 2))
for (mm in methods) {
  hc <- hclust(dist(h_scaled), method = mm)
  plot(hc,
    main = paste("Linkage:", mm), sub = "", xlab = "",
    cex = 0.55, labels = FALSE
  )
}
```

```
par(mfrow = c(1, 1))
```

How to read these four dendrograms:

- **Single linkage:** chains clusters together by their closest pair — tends to produce long, “stringy” clusters.
- **Complete linkage:** merges clusters by their *farthest* pair — produces compact, roughly equal-size balls.
- **Average linkage:** a compromise — the average distance between all pairs.
- **Ward:** minimises the increase in within-cluster variance — the most popular default for numeric data.

The choice is engineering judgement: if you want equal-size planning zones, use Ward. If you want to detect chained anomalies, single linkage exposes them.

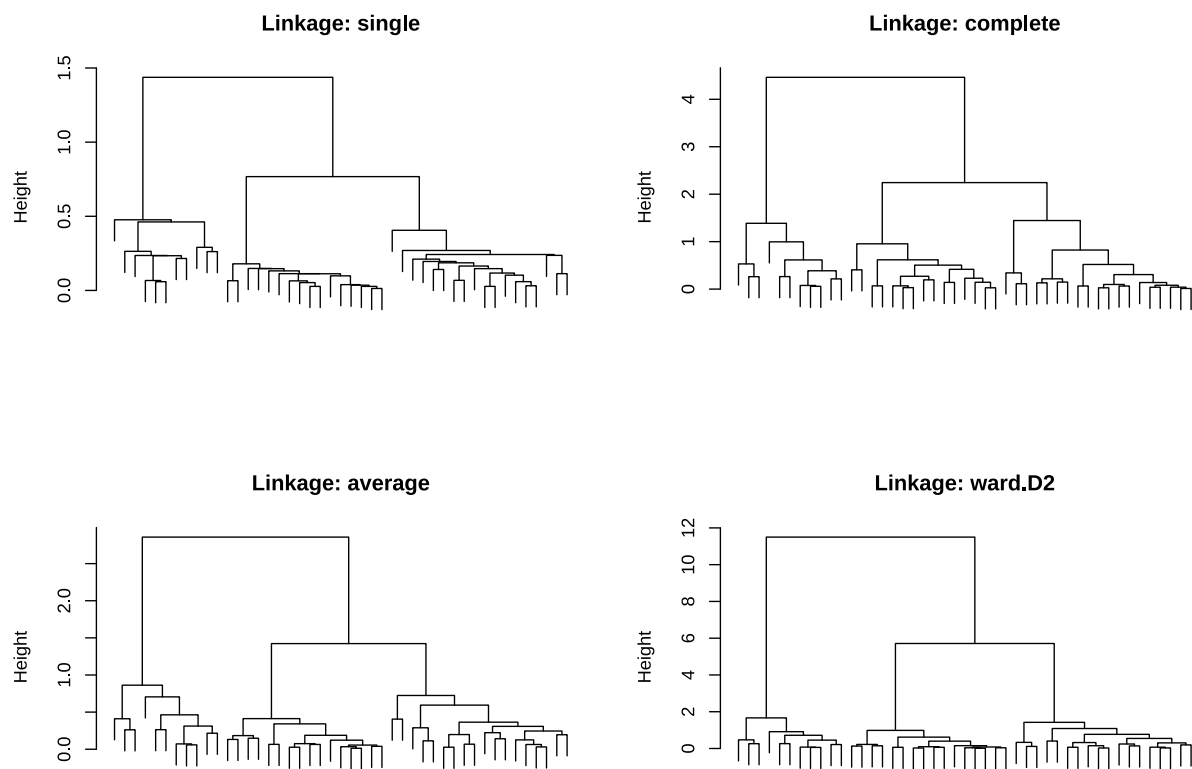


Figure 27: Single linkage chains; complete cuts compact balls; Ward balances sizes. Choice of linkage is an engineering judgement.

```
h_data$hc_cluster <- factor(cutree(hc_ward, k = 3))
ggplot(h_data, aes(pop_density, ndvi, colour = hc_cluster)) +
  geom_point(size = 3) +
  scale_colour_viridis_d(name = "Cluster") +
  labs(
    title = "Settlements grouped by hierarchical clustering",
    x = "Population density (per km2)", y = "NDVI"
  )
```

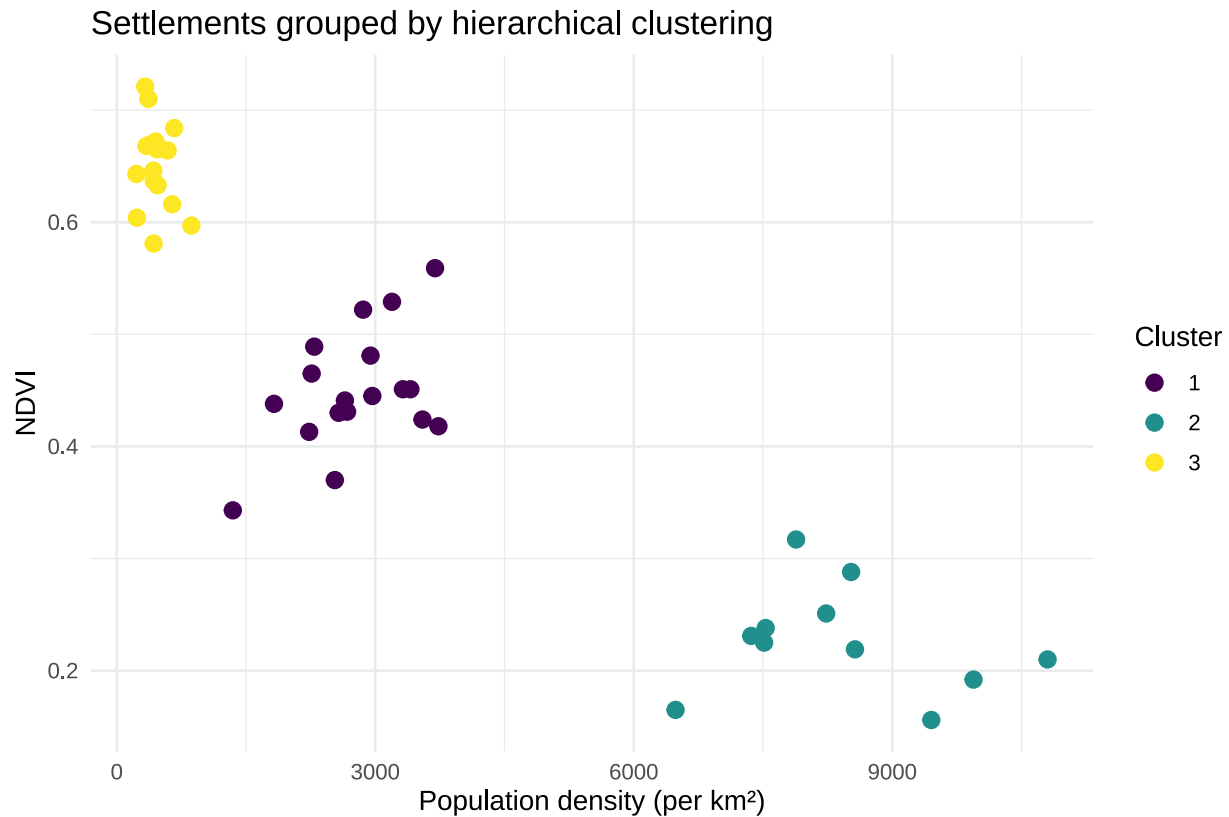


Figure 28: The 45 settlements coloured by hierarchical clusters — same geography as K-means, different route.

How to read this figure: Same as the K-means plot, but the clusters come from hierarchical clustering instead. On a dataset this clean, both methods tend to find the same three zones. On messier data they can disagree — which is a signal to examine the borderline cases manually.

7 Geospatial Data Structures

7.1 Raster vs Vector — The Two Languages of Maps

	Raster	Vector
Model	Grid of equal cells, each holding a value	Points, lines, polygons with coordinates
Examples	Satellite images, DEMs, rainfall surfaces	Pipes, roads, parcel boundaries, boreholes
Resolution	Cell size (e.g., 30 m Landsat, 10 m Sentinel)	Unlimited (true coordinates)
Strengths	Continuous surfaces, image processing, fast overlay	Precise geometry, topology, attributes per feature
In R	terra (SpatRaster)	sf (simple features)

```

grid_df <- expand.grid(x = 1:8, y = 1:8)
set.seed(91)
grid_df$value <- sample(1:5, nrow(grid_df),
  replace = TRUE,
  prob = c(.35, .3, .2, .1, .05)
)
p_raster <- ggplot(grid_df, aes(x, y, fill = factor(value))) +
  geom_tile(colour = "white", linewidth = 0.6) +
  scale_fill_viridis_d(option = "mako", guide = "none") +
  coord_equal() +
  theme_void() +
  labs(
    title = "RASTER - grid of cells",
    subtitle = "satellite images, DEMs, rainfall surfaces"
  )

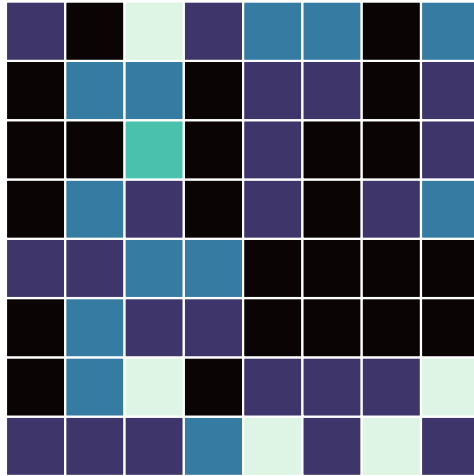
pts_df <- tibble(x = c(2.2, 3.5, 5.1, 6.8), y = c(2.5, 4.2, 3.1, 5.5))
line_df <- tibble(
  x = seq(1, 7, length.out = 60),
  y = 1.2 + 0.35 * sin(seq(1, 7, length.out = 60) * 1.4)
)
poly_df <- tibble(
  x = c(5.2, 7.2, 7.6, 5.6, 5.2),
  y = c(5.6, 5.9, 7.0, 6.6, 5.6)
)
p_vector <- ggplot() +
  geom_polygon(
    data = poly_df, aes(x, y), fill = "palegreen3",
    alpha = 0.85, colour = "grey20"
  ) +
  geom_path(data = line_df, aes(x, y), colour = "steelblue", linewidth = 1.4) +
  geom_point(data = pts_df, aes(x, y), colour = "firebrick", size = 3) +
  coord_equal(xlim = c(1, 8), ylim = c(1, 8)) +
  theme_void() +
  labs(
    title = "VECTOR - points, lines, polygons",
    subtitle = "wells, pipelines, parcels"
  )

p_raster | p_vector

```

How to read the two panels:

RASTER — grid of cells
satellite images, DEMs, rainfall surfaces



VECTOR — points, lines, polygons
wells, pipelines, parcels

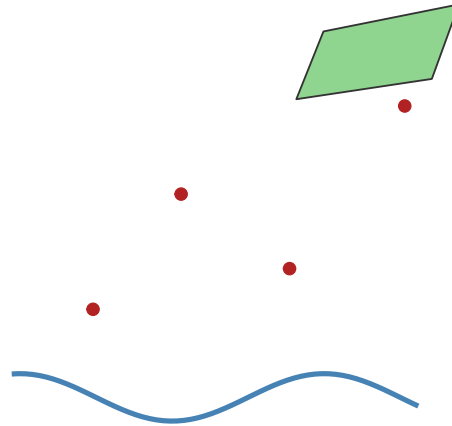


Figure 29: Left: raster = cells. Right: vector = points (wells), line (pipe), polygon (parcel).

- **Left (Raster):** The world is chopped into a grid of equal cells, each holding one value. Satellite images, elevation models, and rainfall surfaces are all rasters.
- **Right (Vector):** The world is described by precise geometry — points (wells), lines (pipes), and polygons (parcels). Each feature can carry its own attributes.

In practice you will use both. A flood susceptibility workflow might use a DEM raster, a river polyline vector, and a set of observed-flood point locations. `terra` handles the first; `sf` handles the other two.

7.2 Rasters in R with terra

```
set.seed(92)
r100 <- rast(
  nrows = 50, ncols = 50, xmin = 0, xmax = 5000,
  ymin = 0, ymax = 5000, crs = "EPSG:32643"
)
xy_r <- xyFromCell(r100, 1:ncell(r100))
elev_r <- 200 + 100 * sin(xy_r[, "x"] / 1500) + 60 * cos(xy_r[, "y"] / 1200) +
  rnorm(nrow(xy_r), 0, 5)
values(r100) <- elev_r
names(r100) <- "elevation_m"
plot(r100, col = viridis(100), main = "DEM raster - 100 m cells")
```

How to read this map: The colour scale shows elevation from low (dark) to high (yellow). Each grid cell is 100 m × 100 m. The `crs = "EPSG:32643"` sets the coordinate reference system (UTM zone 43N) — the layer is geographically anchored. **This is the kind of object you would download from Bhuvan or USGS and immediately read into R with `rast("dem.tif")`.**

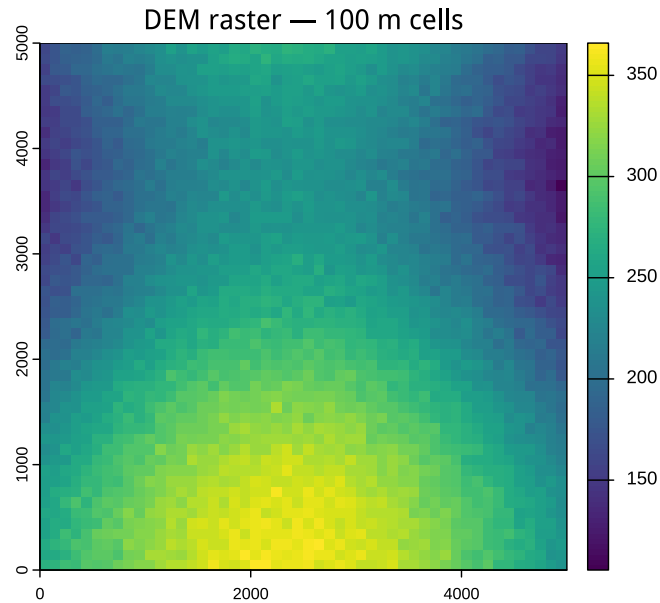


Figure 30: A digital elevation model as a SpatRaster (100 m cells).

7.2.1 Resolution matters — coarsening the mesh

```
r25 <- aggregate(r100, fact = 2) # 200 m cells
par(mfrow = c(1, 2))
plot(r100, col = viridis(100), main = "100 m cells (fine)")
plot(r25, col = viridis(100), main = "200 m cells (coarse)")
```

```
par(mfrow = c(1, 1))
```

How to read this comparison: The left raster has 100 m cells; the right has 200 m cells (each new cell is the average of 4 old ones). The coarser version is faster to process and smaller to store, but loses fine detail — exactly like using a coarser mesh in FEA. **Choose resolution based on what decisions you need to support**, not on what looks nicest.

7.3 Vectors in R with sf

```
wells <- st_as_sf(
  tibble(
    name = c("W1", "W2", "W3", "W4"),
    x = c(1000, 2500, 3600, 4400),
    y = c(1200, 3000, 2200, 4100)
  ),
  coords = c("x", "y"), crs = 32643
)

pipe_line <- st_sfc(
```

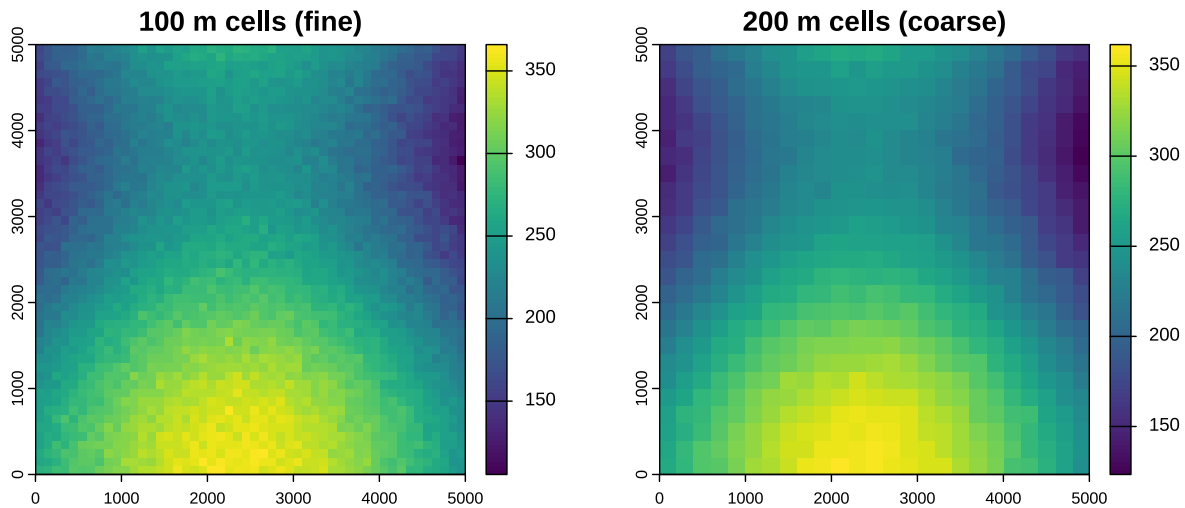


Figure 31: Aggregating a raster = coarsening the mesh. Fine cells capture detail; coarse cells are faster but blurrier.

```

st_linestring(cbind(c(200, 1800, 3200, 4800), c(4000, 3200, 3600, 1000))),
crs = 32643
)

building <- st_sfc(
  st_polygon(list(cbind(
    c(2600, 3400, 3400, 2600, 2600),
    c(1200, 1200, 2000, 2000, 1200)
  ))),
  crs = 32643
)

ggplot() +
  geom_sf(data = building, fill = "wheat2", colour = "grey30") +
  geom_sf(data = pipe_line, colour = "steelblue", linewidth = 1.2) +
  geom_sf(data = wells, shape = 21, fill = "firebrick", size = 3.2) +
  geom_sf_text(data = wells, aes(label = name), nudge_y = 200, size = 3.4) +
  labs(
    title = "Vector data with sf",
    subtitle = "points = wells | line = pipeline | polygon = building"
  ) +
  theme_void()

```

How to read this vector map: All three geometry types are shown together — points (wells, red dots), lines (pipeline, blue curve), and a polygon (building, tan). Each carries a **CRS**, without which geometry is meaningless. `geom_sf()` is the ggplot2 bridge that draws any `sf` object without extra work.

A taste of spatial analysis in one line — distance of each well to the pipeline (useful for **right-of-way compliance**):

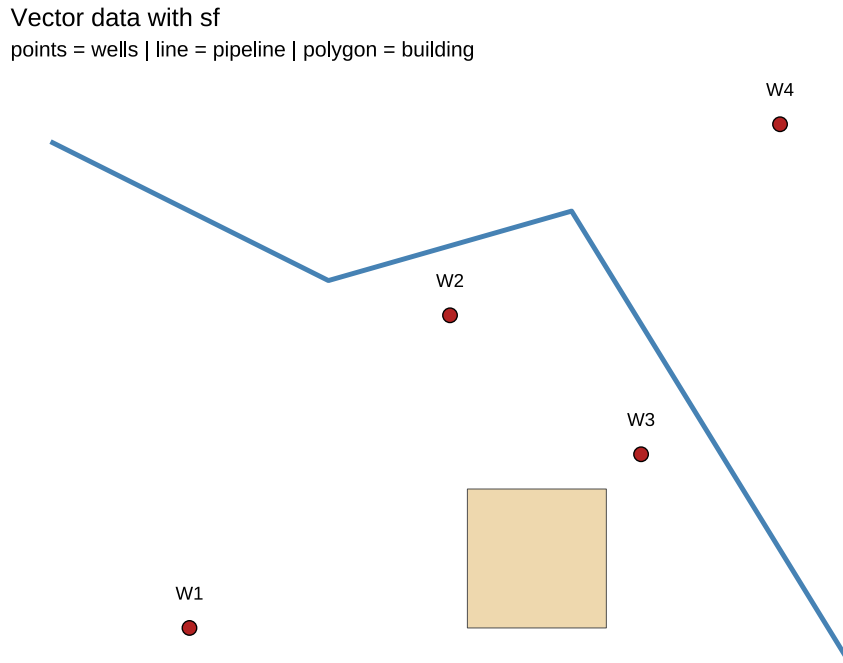


Figure 32: An sf map: monitoring wells (points), a water pipeline (line), a building footprint (polygon).

```
st_distance(wells, pipe_line)
```

```
## Units: [m]
##      [,1]
## [1,] 2154.0659
## [2,]  384.6096
## [3,]  393.0731
## [4,] 1284.0387
```

How to read this output: A matrix of distances in metres (because the CRS is a metric projection, not lat/lon degrees). Each row = one well; the single column = distance to the pipeline. **Real use:** the well with the smallest distance is the one that violates right-of-way and needs to be relocated.

Real data at home: read a GeoTIFF with `rast("dem.tif")`, a shapefile or GeoPackage with `st_read("roads.shp")`. Free sources: Bhuvan (ISRO), USGS EarthExplorer, Copernicus Open Access, OpenStreetMap.

8 Satellite Image Processing

8.0.1 Simulate a 4-band image (Red, Green, Blue, NIR)

Each land-cover type has a **spectral signature** — the key physics: **vegetation reflects NIR strongly** (leaf structure) but absorbs red (photosynthesis). That contrast is what NDVI exploits.

```

set.seed(101)
nr <- 100
nc <- 100
img <- rast(
  nrows = nr, ncols = nc, xmin = 0, xmax = 1000,
  ymin = 0, ymax = 1000, nlyrs = 4
)
xy_img <- xyFromCell(img, 1:ncell(img))
colnames(xy_img) <- c("x", "y")

river_c <- 420 + 110 * sin(xy_img[, "x"] / 140)
dist_riv <- abs(xy_img[, "y"] - river_c)

land_class <- ifelse(dist_riv < 28, "water",
  ifelse(xy_img[, "x"] > 620 & xy_img[, "y"] > 620, "urban",
    ifelse(rnorm(ncell(img), 0, 45) < -22, "bare", "vegetation")
  )
)

sig_mat <- matrix(
  c( # rows = bands, cols = classes
    .06, .08, .30, .25, # Red
    .09, .16, .34, .25, # Green
    .13, .08, .30, .25, # Blue
    .03, .46, .38, .21
  ), # NIR
  nrow = 4, byrow = TRUE,
  dimnames = list(
    c("red", "green", "blue", "nir"),
    c("water", "vegetation", "bare", "urban")
  )
)

set.seed(102)
band_mat <- t(sig_mat[, land_class]) +
  matrix(rnorm(4 * ncell(img), 0, 0.015), ncol = 4, byrow = TRUE)

values(img) <- band_mat
names(img) <- c("red", "green", "blue", "nir")

```

What you are seeing: A 4-layer raster with 10 000 cells. Each cell has a red, green, blue, and near-infrared reflectance value. The dataset was built by assigning each cell one of four land-cover classes (water, vegetation, bare soil, urban), then giving it the appropriate spectral signature. **Real satellite imagery works exactly this way** — bands are reflectance values, not colours.

```

par(mfrow = c(1, 2))
plotRGB(img,
  r = 1, g = 2, b = 3, stretch = "lin",
  main = "True colour (R, G, B)"
)
plotRGB(img,
  r = 4, g = 1, b = 2, stretch = "lin",
  main = "False colour (NIR, R, G)"
)

```

)

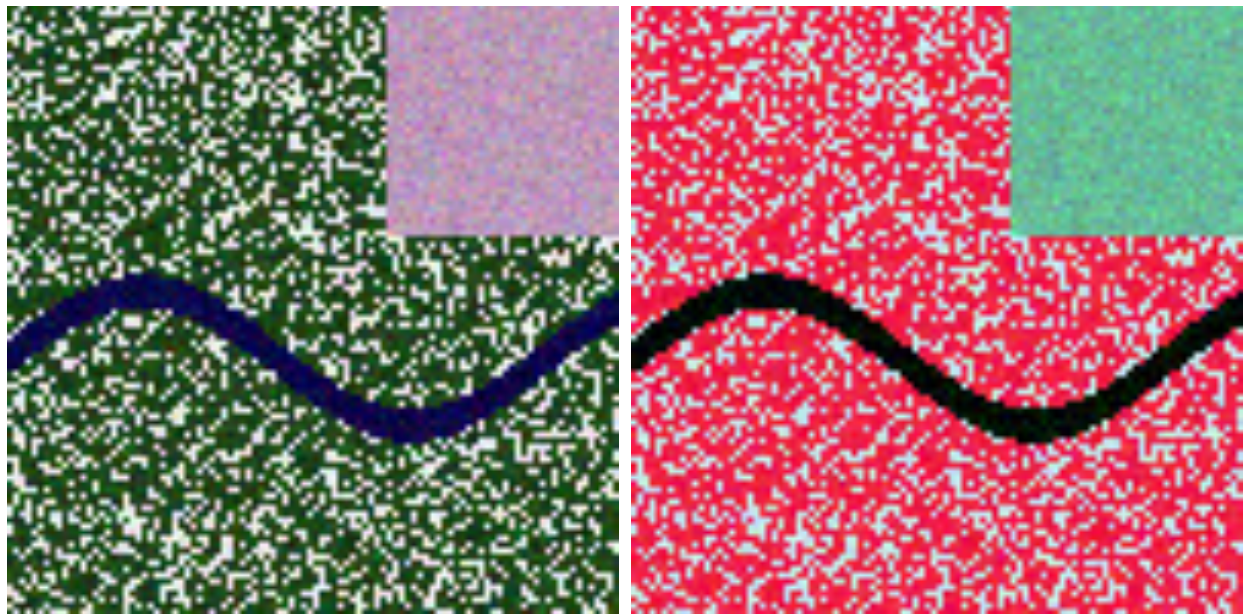


Figure 33: True colour (left) vs false colour composite (right): healthy vegetation glows red in the NIR band.

```
par(mfrow = c(1, 1))
```

How to read the two composites:

- **True colour (left):** Bands 1, 2, 3 mapped to R, G, B — what your eyes would see. Water is dark blue-black, vegetation is greenish, urban is grey.
- **False colour (right):** Band 4 (NIR) mapped to Red. Because vegetation strongly reflects NIR, **all healthy vegetation glows red** in this composite. This is the standard trick every remote-sensing analyst uses to make vegetation pop out visually.

8.0.2 NDVI — the most-used vegetation index on Earth

$$NDVI = \frac{NIR - Red}{NIR + Red}$$

```
ndvi <- (img$nir - img$red) / (img$nir + img$red)
plot(ndvi,
     col = viridis(100),
     main = "NDVI = (NIR - Red) / (NIR + Red)"
)
```

How to read the NDVI map: Yellow-green areas = high NDVI = healthy vegetation. Dark areas = low NDVI = water, bare soil, or built-up surfaces. The river corridor running across the image shows up as a dark line. **Values range from -1 to $+1$** by definition, but real land surfaces almost always sit between 0 and 1.

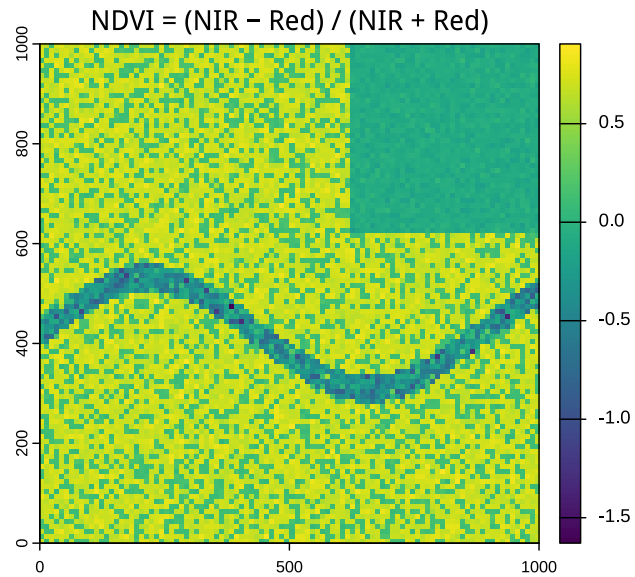


Figure 34: NDVI map: water 0 or below, bare soil 0.1-0.2, vegetation > 0.3.

```
ndvi_vals <- values(ndvi)[, 1]
tibble(ndvi = ndvi_vals) %>%
  ggplot(aes(ndvi)) +
  geom_histogram(bins = 50, fill = "forestgreen", colour = "white") +
  geom_vline(xintercept = 0.3, linetype = "dashed", colour = "firebrick") +
  labs(
    title = "NDVI distribution - thresholding is a 1-class classifier",
    x = "NDVI", y = "Cell count"
  )
```

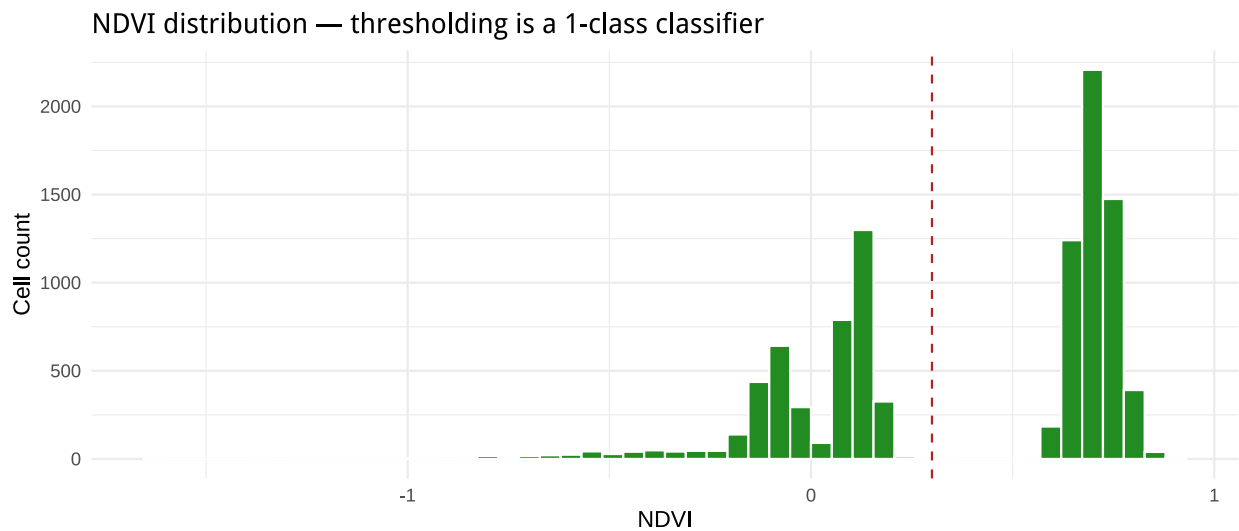


Figure 35: NDVI histogram: the dashed line (0.3) is a simple threshold rule to build a vegetation mask.

```

rcl_v <- matrix(c(
  -1, 0.3, 0,
  0.3, 1, 1
), ncol = 3, byrow = TRUE)
veg_mask <- classify(ndvi, rcl_v)
plot(veg_mask,
  col = c("grey85", "forestgreen"),
  main = "Vegetation mask (NDVI > 0.3)"
)

```

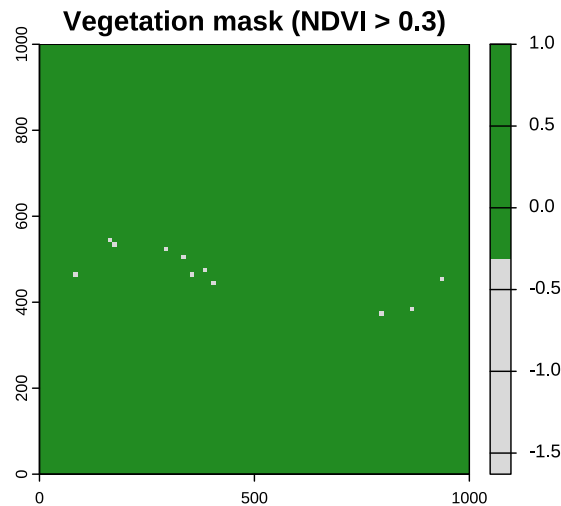


Figure 36: NDVI histogram: the dashed line (0.3) is a simple threshold rule to build a vegetation mask.

How to read the histogram and mask: The histogram shows the distribution of NDVI across all cells. The dashed line at 0.3 is a *chosen threshold* — anything above it is declared vegetation. **The resulting binary map (bottom) is a classifier** — a very simple one, but a classifier nonetheless. Real ML work often begins with such a threshold as a baseline before trying something smarter.

Where you will use this: crop area estimation (agriculture engineering), pre/post-monsoon greenness change (flood damage), corridor vegetation clearance auditing (highways), urban green-space equity (planning). Real missions: **Landsat 8/9 (30 m)**, **Sentinel-2 (10 m, free)**.

9 Remote Sensing Integration: GIS + AI Fusion

9.1 The Workflow That Runs Modern Geospatial ML

```

steps <- tibble(
  x = 1:5,
  label = c(
    "Geospatial layers\n(raster + vector)",
    "extract() features\nat training points",
  )
)

```

```

    "Machine learning\nmodel (e.g., RF)",
    "predict() on the\nwhole raster",
    "Susceptibility map\n→ decisions"
  ),
  fill = c("#4C72B0", "#55A868", "#C44E52", "#8172B2", "#CCB974")
)
ggplot() +
  geom_rect(
    data = steps, aes(
      xmin = x - 0.42, xmax = x + 0.42,
      ymin = 0, ymax = 1.4, fill = fill
    ),
    colour = "grey25"
  ) +
  geom_text(
    data = steps, aes(x, y = 0.7, label = label), size = 3,
    fontface = "bold", colour = "white", lineheight = 0.9
  ) +
  geom_segment(
    data = tibble(x = 1:4, xend = 2:5),
    aes(x = x + 0.45, xend = xend - 0.45, y = 0.7, yend = 0.7),
    arrow = arrow(length = unit(0.2, "cm")),
    linewidth = 0.8, colour = "grey30"
  ) +
  scale_fill_identity() +
  xlim(0.3, 5.7) +
  ylim(-0.2, 1.6) +
  theme_void() +
  labs(title = "GIS + AI fusion in five steps")

```

GIS + AI fusion in five steps

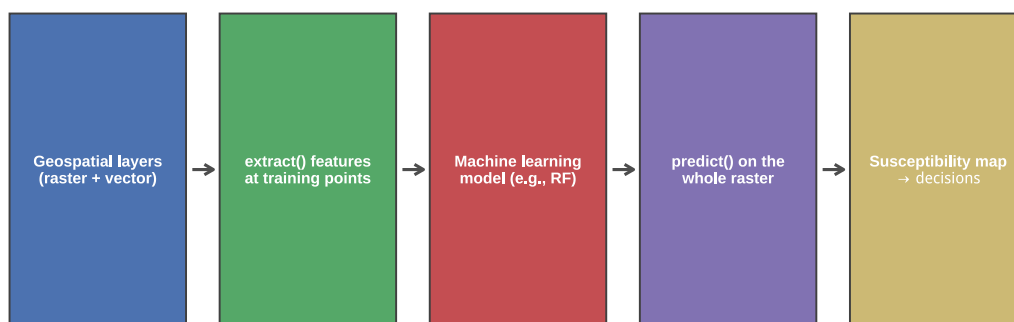


Figure 37: GIS + AI fusion: extract table-like features from map layers, learn, then predict back onto the map.

How to read the workflow: Five steps, left to right. **Step 1:** gather raster and vector layers (DEM, rainfall, NDVI, rivers). **Step 2:** at each labelled training point, extract the pixel value of every layer into a table row — this is the map→table translator. **Step 3:** train any ML classifier on the table. **Step 4:** feed the *entire* raster stack back to the trained model — this is the table→map translator. **Step 5:** classify the

resulting probabilities into planning zones.

The fusion secret: a GIS sees a *map*; an ML algorithm sees a *table*. `terra::extract()` is the translator **map** $\rightarrow$ **table**; `terra::predict()` is the translator **table** $\rightarrow$ **map**. Learn these two functions and you can attach any classifier to any landscape.

9.2 Full Demonstration: Flood Susceptibility Mapping End-to-End

9.2.1 Step 1 — build the geospatial knowledge base (4 terrain layers)

```
set.seed(111)
terr <- rast(
  nrows = 80, ncols = 80, xmin = 0, xmax = 8000,
  ymin = 0, ymax = 8000, nlyrs = 4
)
xy_t <- xyFromCell(terr, 1:ncell(terr))
colnames(xy_t) <- c("x", "y")

elevation_t <- 250 + 130 * sin(xy_t[, "x"] / 2600) +
  90 * cos(xy_t[, "y"] / 2100) + rnorm(nrow(xy_t), 0, 4)
slope_t <- pmax(0.5, 6 + 7 * sin(xy_t[, "x"] / 1700) *
  cos(xy_t[, "y"] / 1500) + rnorm(nrow(xy_t), 0, 0.6))
rain_t <- 950 + 320 * sin((xy_t[, "x"] + xy_t[, "y"]) / 4200) +
  rnorm(nrow(xy_t), 0, 15)
ndvi_t <- pmin(pmax(0.62 - 0.00004 * xy_t[, "x"] -
  0.00002 * xy_t[, "y"] +
  rnorm(nrow(xy_t), 0, 0.06), 0.05), 0.9)

values(terr) <- cbind(elevation_t, slope_t, rain_t, ndvi_t)
names(terr) <- c("elevation", "slope", "rainfall", "ndvi")
plot(terr)
```

How to read the four panels: Each panel is one environmental layer across the same 8×8 km landscape. *elevation* has smooth hills; *slope* varies sinusoidally; *rainfall* has a gradient; *ndvi* (greenness) is highest in the lower-left corner. **These four rasters together are the model's inputs** — every cell will be a training or prediction example.

9.2.2 Step 2 — extract features at 400 training points (map $\rightarrow$ table)

```
set.seed(112)
n_train <- 400
cells_t <- sample(ncell(terr), n_train)
pts_xy <- as_tibble(xyFromCell(terr, cells_t))
env_t <- as_tibble(extract(terr, cells_t))

# simulate field observations of flooding
logit_t <- 5 - 0.020 * env_t$elevation - 0.25 * env_t$slope +
  0.004 * env_t$rainfall - 2.0 * env_t$ndvi
flood_obs <- factor(ifelse(rbinom(n_train, 1, plogis(logit_t)) == 1,
  "yes", "no"))
```

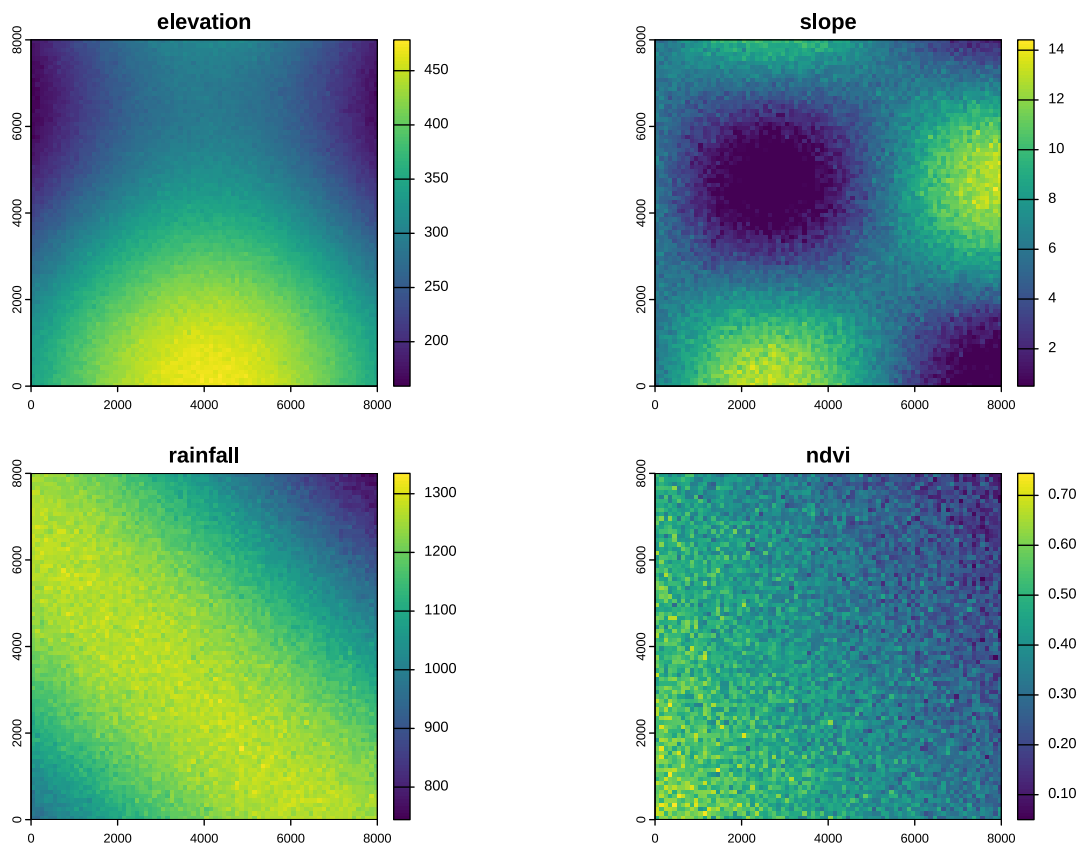


Figure 38: The four environmental layers that will become ML features.

```
), levels = c("no", "yes"))

train_tab <- bind_cols(pts_xy, env_t, tibble(flood = flood_obs))
head(train_tab)
```

```
## # A tibble: 6 x 7
##       x       y elevation slope rainfall  ndvi flood
##   <dbl> <dbl>   <dbl> <dbl>   <dbl> <dbl> <fct>
## 1  2950  7850     292.  8.81    1118.  0.312 yes
## 2  4150  4150     343.  2.31    1233.  0.500 no
## 3   550  4750     227.  4.97    1259.  0.552 yes
## 4  2350  7550     270.  8.77    1174.  0.432 yes
## 5  3550  5950     291.  2.03    1219.  0.366 yes
## 6  4750   50     473.  7.88    1236.  0.470 no
```

How to read this table: Each row is one sampled point on the map, with its x/y coordinates, the extracted value of every environmental layer at that point, and the observed flood label. **This table is what the ML model sees.** The map has disappeared; only numbers remain. The same table shape works whether you have 400 points or 40,000.

9.2.3 Step 3 — train the model on the table

```
rf_terr <- randomForest(flood ~ elevation + slope + rainfall + ndvi,
  data = train_tab, ntree = 500
)
rf_terr
```

```
##
## Call:
## randomForest(formula = flood ~ elevation + slope + rainfall +      ndvi, data = train_tab, ntree = 500)
##               Type of random forest: classification
##               Number of trees: 500
## No. of variables tried at each split: 2
##
##               OOB estimate of  error rate: 24.5%
## Confusion matrix:
##      no yes class.error
## no  53  67   0.5583333
## yes 31 249   0.1107143
```

How to read the printed forest: Standard random-forest output — number of trees, OOB confusion matrix, and per-class OOB error. The low OOB error tells you the model has learned a real signal. **This is the same output format you saw in Section 5** — the workflow generalises: the terrain layers just became columns of a table.

9.2.4 Step 4 — predict on the whole raster (table → map)

```

pred_fun <- function(model, newdata) {
  predict(model, newdata = newdata, type = "prob")[, "yes"]
}
flood_prob <- predict(terr, rf_terr, fun = pred_fun, na.rm = TRUE)

plot(flood_prob,
     col = hcl.colors(100, "Spectral", rev = TRUE),
     main = "Predicted flood susceptibility (GIS + ML fusion)"
)
points(xy_t[cells_t, ],
      col = ifelse(flood_obs == "yes", "black", "grey60"),
      pch = 19, cex = 0.5
)
legend("topright", c("Observed flood", "No flood"),
      col = c("black", "grey60"), pch = 19, cex = 0.8, bty = "n"
)

```

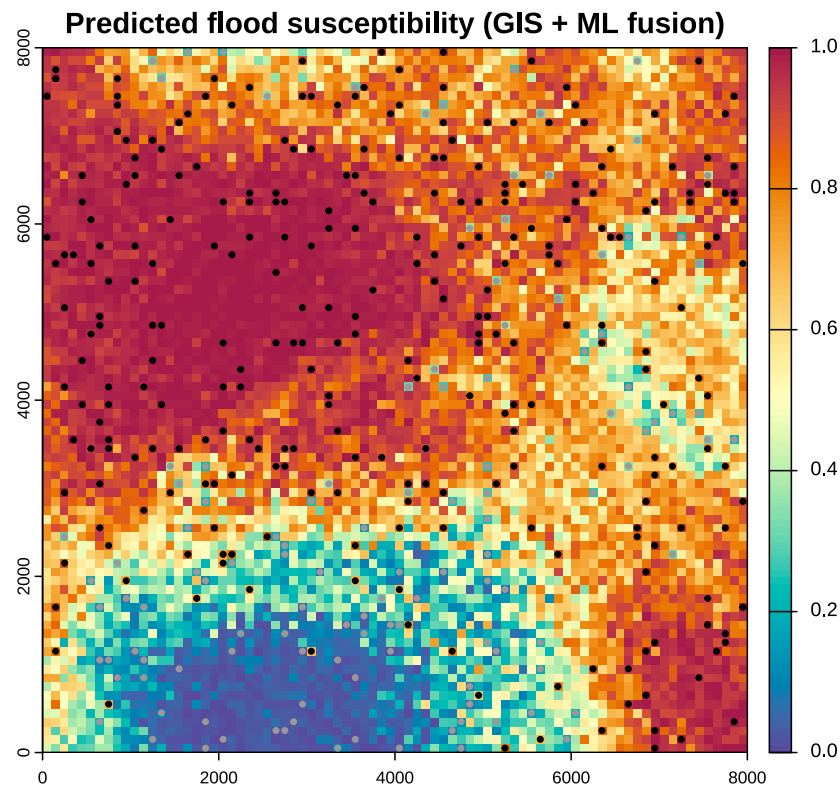


Figure 39: The final susceptibility map — every cell of the landscape scored by the forest. Black dots = observed flood points.

How to read this map: The colour scale runs from low (blue) to high (red) predicted flood probability. **Every pixel of the landscape now has a prediction** — even those we never sampled. Black dots are observed flood points; grey are non-flood. A well-trained model places black dots predominantly in the red/yellow zones and grey dots in the blue zones.

9.2.5 Step 5 — convert probabilities into planning zones

```
rcl_f <- matrix(c(
  0, 0.2, 1,
  0.2, 0.4, 2,
  0.4, 0.6, 3,
  0.6, 0.8, 4,
  0.8, 1, 5
), ncol = 3, byrow = TRUE)
flood_class <- classify(flood_prob, rcl_f)
plot(flood_class,
  col = c("lightblue", "skyblue", "khaki", "orange", "firebrick"),
  main = "Flood susceptibility zones (1 = very low ... 5 = very high)"
)
```

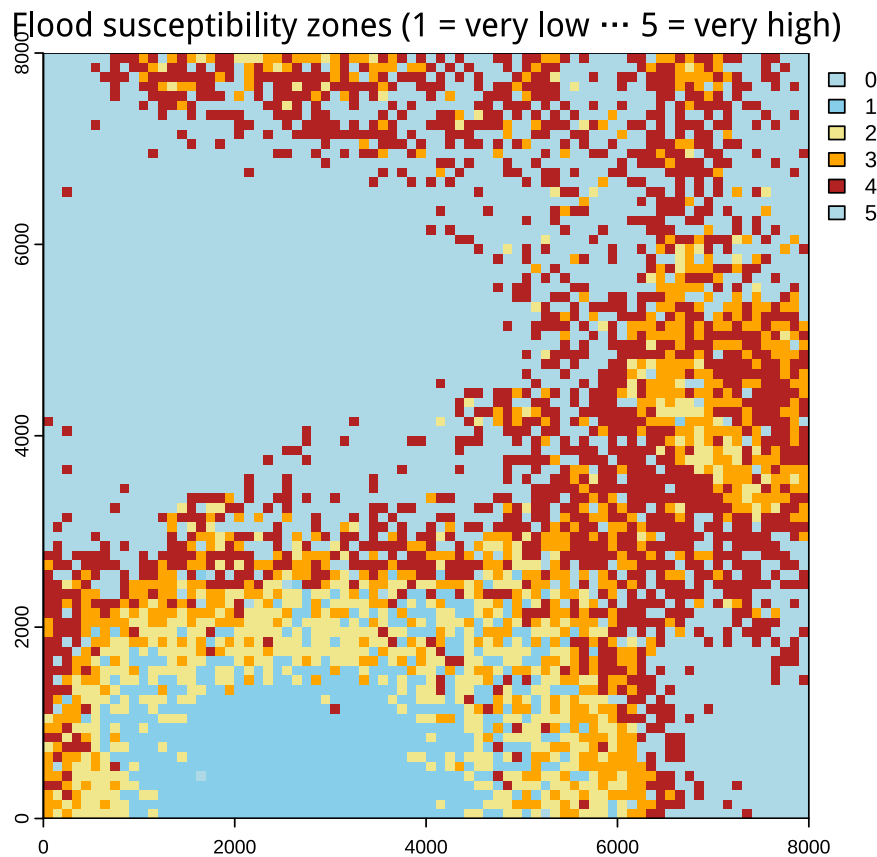


Figure 40: Zoned susceptibility map — this is the layer that goes into the land-use plan.

How to read this zoned map: The continuous probability map has been collapsed into five discrete classes — very low, low, moderate, high, very high. **Discrete classes are what policy and planning need.** A town plan says “no construction in zone 5,” not “no construction where probability > 0.83.” The thresholds (0.2, 0.4, 0.6, 0.8) are an engineering choice — you can also use quantile breaks for equal-area zones.

You have just performed the exact workflow of published flood/landslide susceptibility papers

— only with simulated data. Swap the simulated stack for real Sentinel-2 + DEM derivatives and the code barely changes.

10 Advanced Mapping Applications

10.1 Urban Heat Island (UHI) Analysis

The phenomenon: cities replace vegetation and water with concrete and asphalt → they absorb more solar radiation and evaporate less → the city core runs several degrees hotter than its rural surroundings.

Civil engineering relevance: pavement materials, building density, and green-cover mandates are *design levers* on city temperature. UHI studies guide cool-roof codes, park placement, and heat-action plans.

```
set.seed(121)
nr_u <- 120
nc_u <- 120
uhi <- rast(
  nrows = nr_u, ncols = nc_u, xmin = 0, xmax = 12000,
  ymin = 0, ymax = 12000, nlyrs = 2
)
xy_u <- xyFromCell(uhi, 1:ncell(uhi))
colnames(xy_u) <- c("x", "y")
d_center <- sqrt((xy_u[, "x"] - 6000)^2 + (xy_u[, "y"] - 6000)^2)

ndvi_u <- pmin(pmax(0.7 - 0.65 * exp(-d_center^2 / (2 * 2600^2)) +
  rnorm(nrow(xy_u), 0, 0.05), 0.02), 0.9)
lst_u <- 30 + 8.5 * exp(-d_center^2 / (2 * 2500^2)) -
  6 * (ndvi_u - 0.3) + rnorm(nrow(xy_u), 0, 0.4)

values(uhi) <- cbind(lst_u, ndvi_u)
names(uhi) <- c("LST_C", "NDVI")

plot(uhi$LST_C,
  col = hcl.colors(100, "Inferno"),
  main = "Land Surface Temperature (°C) - urban heat island"
)
```

How to read this map: The colour scale is land-surface temperature. Note the **bullseye pattern** — the brightest (hottest) area is the city centre, surrounded by cooler rural pixels. The temperature difference between the two is the **UHI intensity** — typically 3–8 °C in real cities.

10.1.1 The transect: temperature along an east–west line

```
dy <- abs(xy_u[, "y"] - 6000)
idx_row <- which(dy == min(dy))
tran <- tibble(
  x = xy_u[idx_row, "x"],
  lst = values(uhi$LST_C)[idx_row, 1]
```

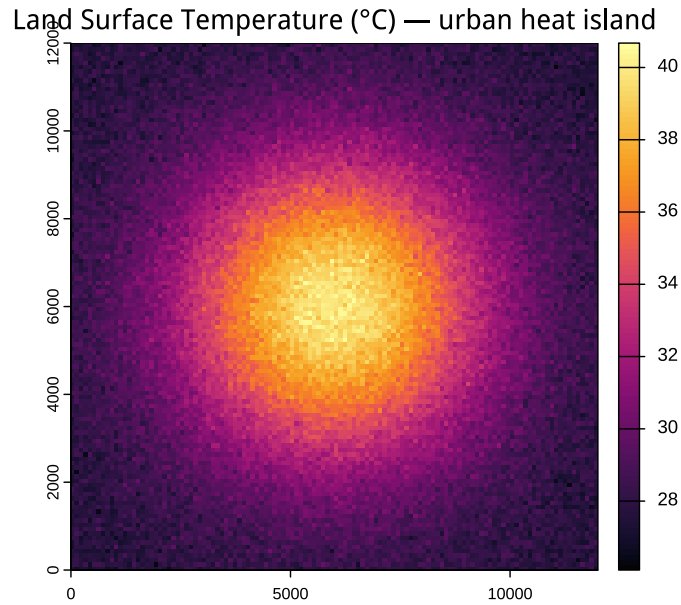


Figure 41: Simulated land-surface temperature: a hot core over a cooler rural halo — the classic UHI bullseye.

```

) %>%
  group_by(x) %>%
  summarise(lst = mean(lst)) %>%
  arrange(x)

rural_base <- mean(lst_u[d_center > 5500])
uhi_int <- max(lst_u) - rural_base

ggplot(tran, aes(x, lst)) +
  geom_line(colour = "firebrick", linewidth = 1.2) +
  geom_hline(yintercept = rural_base, linetype = "dashed", colour = "forestgreen") +
  annotate("text",
    x = 800, y = rural_base + 0.6, colour = "forestgreen",
    size = 3.4, label = "rural baseline"
  ) +
  annotate("text",
    x = 6000, y = max(tran$lst) - 0.3, size = 3.6, fontface = "bold",
    colour = "firebrick",
    label = sprintf("UHI intensity  %.1f °C", uhi_int)
  ) +
  labs(
    title = "E-W temperature transect through the city",
    x = "Distance from western boundary (m)", y = "LST (°C)"
  )

```

How to read the transect: The curve shows temperature as you walk east across the city. The dashed green line is the rural baseline (average temperature far from the centre). The bump in the middle is the urban heat island. **The peak height above the baseline is the UHI intensity** — the number you would quote to a city planner.

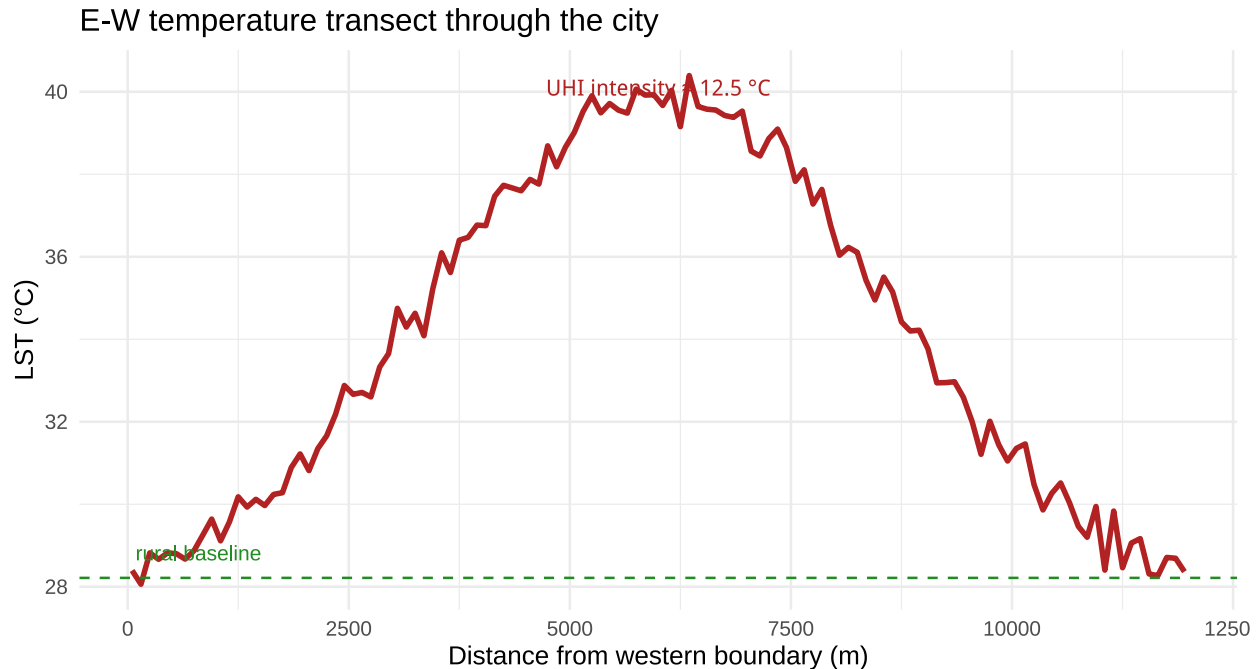


Figure 42: UHI profile across the city — read it like a longitudinal section through a heat 'dome'.

10.1.2 The green-cooling effect: NDVI vs LST

```
tibble(ndvi = ndvi_u, lst = lst_u) %>%
  ggplot(aes(ndvi, lst)) +
  geom_point(alpha = 0.25, colour = "darkgreen", size = 0.8) +
  geom_smooth(method = "lm", colour = "black") +
  labs(
    title = "Green cooling: NDVI vs land surface temperature",
    subtitle = "Regression slope = °C of cooling per unit of NDVI",
    x = "NDVI", y = "LST (°C)"
  )
```

How to read this scatter: Each point is a pixel. The negative slope of the fitted line tells you that greener pixels are cooler pixels. **The slope's magnitude is directly useful:** a slope of -5 means that increasing NDVI by 1 unit cools the surface by 5°C . That number is what lets you estimate how much temperature relief a proposed new park would provide.

Real workflow: download Landsat 8/9 thermal band $\rightarrow$ convert DN to LST (radiance $\rightarrow$ brightness temp $\rightarrow$ emissivity correction) $\rightarrow$ compute NDVI from bands 4/5 $\rightarrow$ reproduce this exact analysis for your own city in QGIS or R.

10.2 Landslide Susceptibility Mapping

The standard practice: build an *inventory* of observed landslides, relate them to *conditioning factors* (slope, rainfall, geology, land cover), fit a model, and extrapolate the hazard over the whole terrain.

Green cooling: NDVI vs land surface temperature

Regression slope = °C of cooling per unit of NDVI

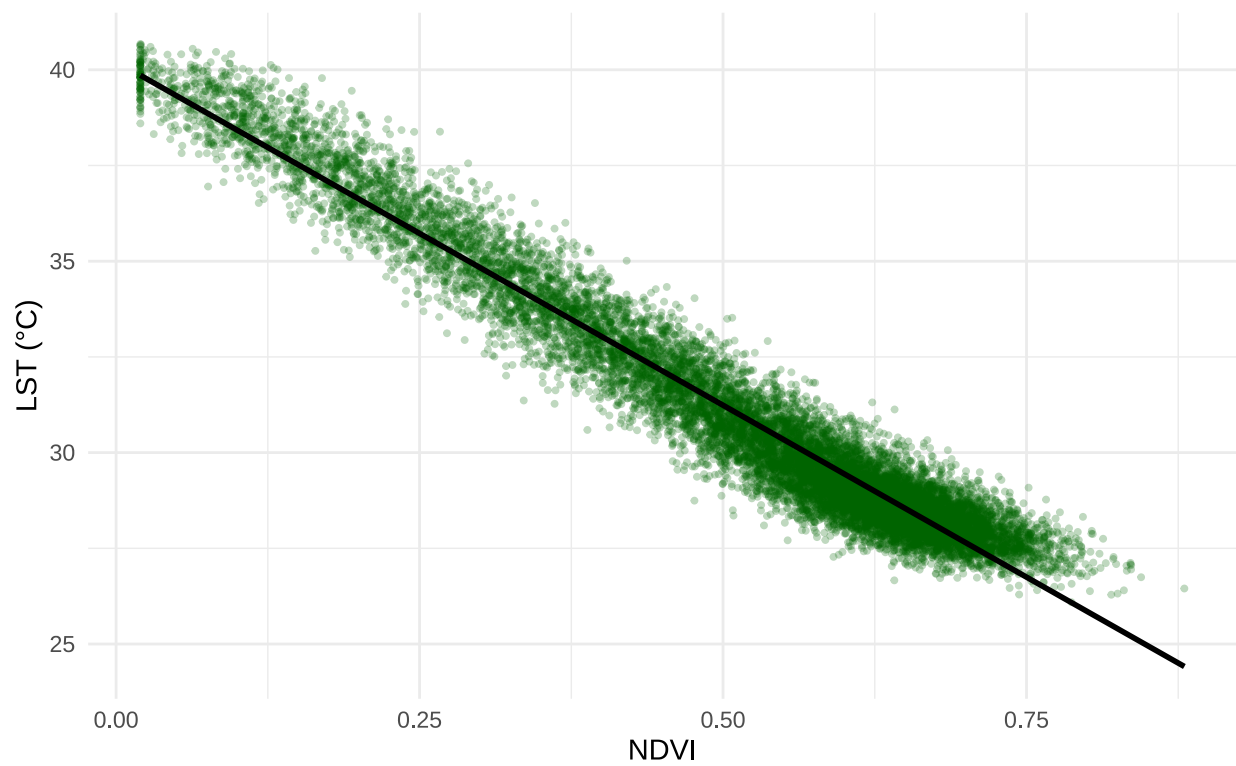


Figure 43: Every unit of vegetation buys measurable cooling — quantify the slope and you can price a new park.

```

set.seed(131)
n_ls <- 500
land <- tibble(
  slope      = round(runif(n_ls, 2, 40), 1),
  rainfall   = round(runif(n_ls, 700, 2200), 0),
  elevation  = round(runif(n_ls, 100, 900), 0),
  ndvi       = round(runif(n_ls, 0.05, 0.85), 2),
  rock_soft  = sample(c(0, 1), n_ls, replace = TRUE, prob = c(0.6, 0.4))
) %>%
mutate(
  logit_ls = -4 + 0.07 * slope + 0.0014 * rainfall -
    0.0012 * elevation - 1.0 * ndvi + 1.2 * rock_soft,
  landslide = factor(ifelse(rbinom(n_ls, 1, plogis(logit_ls)) == 1,
    "yes", "no"
  ), levels = c("no", "yes"))
)
mean(land$landslide == "yes")

```

```
## [1] 0.268
```

What you are seeing: A 500-site landslide inventory. Each site has five conditioning factors and a binary landslide label. The final number is the base rate of landslides — typically 5–15% in real inventories, matching the simulated result.

```

ls_fit <- glm(landslide ~ slope + rainfall + elevation + ndvi + rock_soft,
  family = binomial, data = land
)
tibble(
  term = names(coef(ls_fit)),
  odds_ratio = round(exp(coef(ls_fit)), 3)
)

```

```

## # A tibble: 6 x 2
##   term      odds_ratio
##   <chr>      <dbl>
## 1 (Intercept)  0.01
## 2 slope      1.10
## 3 rainfall    1.00
## 4 elevation   0.998
## 5 ndvi        0.141
## 6 rock_soft   4.66

```

Reading the odds ratios: Each +1% of slope multiplies landslide odds by $\sim e^{0.07} \approx 1.07$; soft rock multiplies them by ~ 3.3 ; dense vegetation (NDVI) *protects* (OR < 1). **Positive OR = the factor increases hazard; negative OR = protective.** This is exactly how susceptibility factor analyses are reported in journals.

10.2.1 Predict the whole terrain

```
ls_prob <- predict(ls_terr, ls_fit, type = "response")
```

```

qs <- quantile(values(ls_prob)[, 1], probs = c(0.2, 0.4, 0.6, 0.8), na.rm = TRUE)
rcl_ls <- matrix(c(
  -1, qs[1], 1,
  qs[1], qs[2], 2,
  qs[2], qs[3], 3,
  qs[3], qs[4], 4,
  qs[4], 1.01, 5
), ncol = 3, byrow = TRUE)
ls_class <- classify(ls_prob, rcl_ls)

par(mfrow = c(1, 2))
plot(ls_prob,
  col = hcl.colors(100, "Spectral", rev = TRUE),
  main = "Landslide susceptibility (probability)"
)
plot(ls_class,
  col = c(
    "palegreen4", "palegreen3", "khaki1",
    "orange", "firebrick"
  ),
  main = "Susceptibility zones (quantile classes)"
)

```

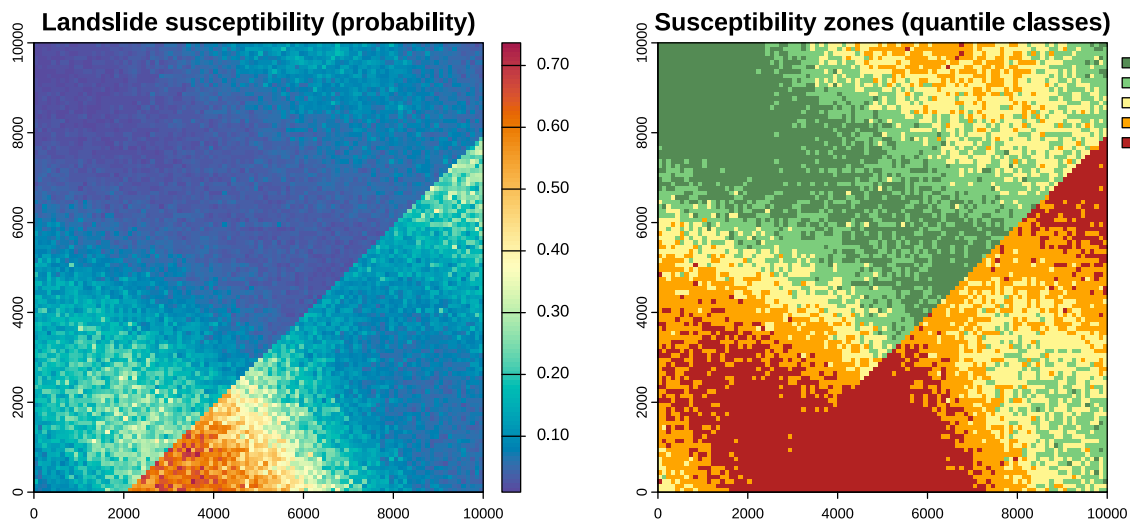


Figure 44: Left: continuous landslide probability. Right: engineering susceptibility zones for planning.

```

par(mfrow = c(1, 1))

```

How to read the pair of maps:

- **Left:** the continuous probability map — every pixel gets a value between 0 and 1.
- **Right:** the same information collapsed into five planning zones using **quantile breaks** (each zone has ~20% of the pixels). The `classify()` function does this in one line.

Quantile zoning ensures equal-area zones. For a hazard map, you might prefer fixed thresholds (e.g., $P > 0.6$ = high) instead — the choice depends on whether you want equal land area per class or equal probability per class.

10.2.2 Area budget per zone (decision support)

```
freq_df <- as.data.frame(freq(ls_class))
cell_area_ha <- (100 * 100) / 10000 # 100 m cells -> 1 ha each
freq_df$area_ha <- freq_df$count * cell_area_ha
freq_df$zone <- c("Very low", "Low", "Moderate", "High", "Very high")[freq_df$value]

ggplot(freq_df, aes(zone, area_ha, fill = zone)) +
  geom_col(colour = "grey25") +
  scale_fill_manual(values = c(
    "palegreen4", "palegreen3", "khaki1",
    "orange", "firebrick"
  ), guide = "none") +
  labs(
    title = "Area by landslide susceptibility class",
    x = NULL, y = expression(Area ~ (ha))
  )
```



Figure 45: How many hectares fall in each susceptibility class — the number the district collector asks for.

How to read this bar chart: Each bar is the **land area** (in hectares) that falls into one susceptibility class. Sum the top two bars to answer the classic policy question: “*How much of the district is high or very high risk?*” That number drives compensation budgets, relocation plans, and building restrictions. **This is the figure that goes into the executive summary of a hazard report.**

10.3 River Change Detection

The task: rivers migrate. Comparing channel positions between two dates quantifies **bank erosion** (land lost) and **deposition** (land gained) — critical for bridges, embankments, and land records.

Civil analogy: overlay of two survey drawings of the same river from two decades — the modern GIS does with `st_difference()` what drafters once did with tracing paper.

```
make_channel <- function(phase = 0, amp = 900, halfw = 150, n = 300) {
  x <- seq(0, 10000, length.out = n)
  y <- 5000 + amp * sin(x / 1800 + phase)
  upper <- cbind(x, y + halfw)
  lower <- cbind(rev(x), rev(y) - halfw)
  st_polygon(list(rbind(upper, lower, upper[1, , drop = FALSE])))
}

r1990 <- st_sfc(make_channel(phase = 0, amp = 900, halfw = 150), crs = 32643)
r2024 <- st_sfc(make_channel(phase = 0.55, amp = 950, halfw = 185), crs = 32643)

erosion <- st_difference(r2024, r1990) # new channel area = bank lost
deposition <- st_difference(r1990, r2024) # abandoned channel = land gained

river_df <- st_sf(year = factor(c("1990", "2024")), geom = c(r1990, r2024))
diff_df <- st_sf(
  type = c(
    "Erosion (2024 channel over 1990 bank)",
    "Deposition (abandoned 1990 channel)"
  ),
  geom = c(erosion, deposition)
)

ggplot() +
  geom_sf(
    data = river_df, aes(fill = year), alpha = 0.3,
    colour = "grey40", linewidth = 0.3
  ) +
  geom_sf(data = diff_df, aes(fill = type), alpha = 0.85, colour = NA) +
  scale_fill_manual(values = c(
    "1990" = "steelblue", "2024" = "darkorange",
    "Erosion (2024 channel over 1990 bank)" = "firebrick",
    "Deposition (abandoned 1990 channel)" = "forestgreen"
  )) +
  labs(
    title = "River migration detection: 1990 → 2024",
    fill = NULL, x = NULL, y = NULL
  ) +
  theme_void()
```

How to read this map: The two channels (1990 and 2024) are overlaid. **Red areas** = land the 2024 channel occupies that the 1990 channel did not — this is **bank erosion** (land lost by landowners). **Green areas** = land the 1990 channel occupied that the 2024 channel abandoned — this is **deposition** (new land formed, often claimed by the state). Where the two overlap, the channel is stable.

River migration detection: 1990 → 2024



Figure 46: River channel 1990 vs 2024 with eroded (red) and deposited (green) areas.

```
tibble(
  zone = c("Stable channel", "Eroded", "Deposited"),
  area_km2 = c(
    as.numeric(st_area(st_intersection(r1990, r2024))) / 1e6,
    as.numeric(st_area(erosion)) / 1e6,
    as.numeric(st_area(deposition)) / 1e6
  )
) %>%
  ggplot(aes(zone, area_km2, fill = zone)) +
  geom_col(colour = "grey25") +
  scale_fill_manual(values = c("steelblue", "firebrick", "forestgreen"), guide = "none") +
  labs(
    title = "River change budget, 1990-2024",
    x = NULL, y = expression(Area ~ (km^2))
  )
```

How to read this chart: Three numbers that sum to the total river-corridor area: stable (both years flooded), eroded (new flood area = land lost), deposited (abandoned flood area = land gained). **For a district administration**, erosion area × land value = compensation budget; deposition area = new revenue land. This is what makes remote sensing pay for itself.

10.3.1 Remote-sensing style change detection with NDVI differencing

```
set.seed(141)
ch <- rast(
```

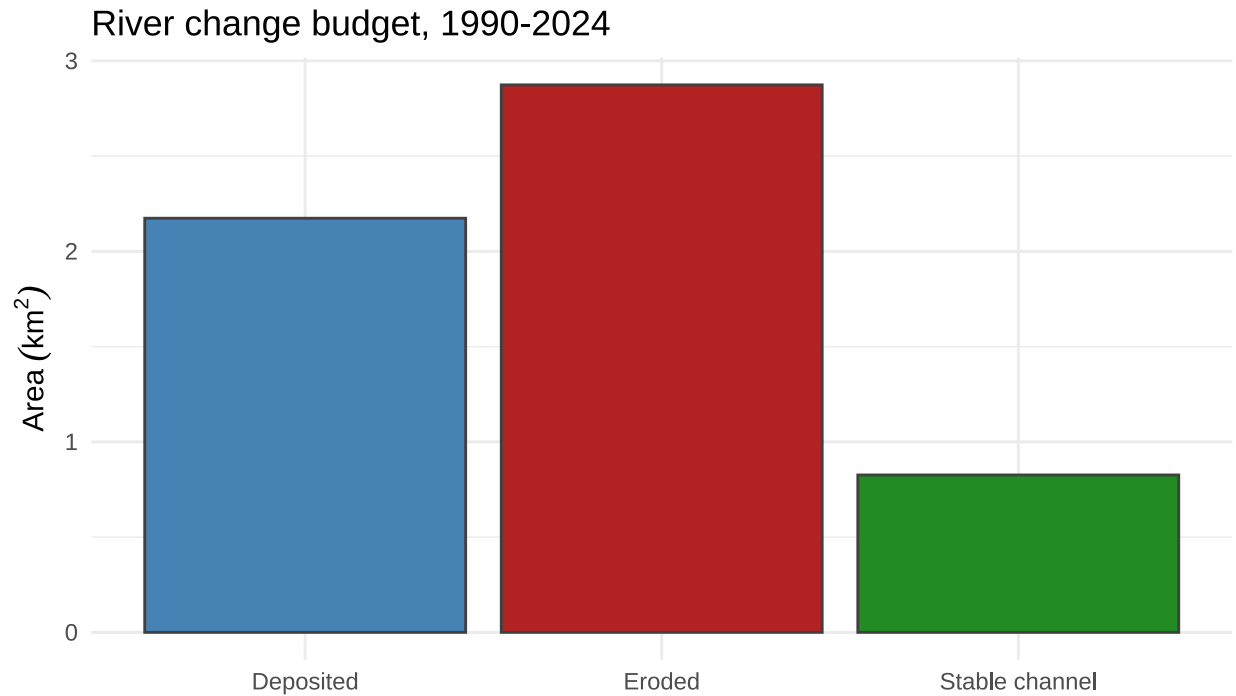


Figure 47: Change budget in km²: the numbers that feed compensation and design decisions.

```

nrows = 60, ncols = 60, xmin = 0, xmax = 6000,
ymin = 0, ymax = 6000, nlyrs = 2
)
xy_c <- xyFromCell(ch, 1:ncell(ch))
colnames(xy_c) <- c("x", "y")

base <- pmin(pmax(0.55 + 0.1 * sin(xy_c[, "x"] / 800) +
  rnorm(nrow(xy_c), 0, 0.06), 0.05), 0.9)
d_riv <- abs(xy_c[, "y"] - (3000 + 500 * sin(xy_c[, "x"] / 1500)))
after <- base
after[d_riv < 250] <- pmax(0.08, base[d_riv < 250] - 0.35) # scour zone

values(ch) <- cbind(base, after)
names(ch) <- c("NDVI_2015", "NDVI_2024")
ndvi_diff <- ch$NDVI_2024 - ch$NDVI_2015

rcl_ch <- matrix(c(
  -1, -0.15, 2,
  -0.15, 0.15, 1,
  0.15, 1, 3
), ncol = 3, byrow = TRUE)
chg <- classify(ndvi_diff, rcl_ch)

par(mfrow = c(1, 2))
plot(ndvi_diff,
  col = hcl.colors(100, "RdYlGn"),
  main = "ΔNDVI 2024 - 2015 (red = vegetation loss)"

```

```
)
plot(chg,
     col = c("grey90", "firebrick", "forestgreen"),
     main = "Classes: grey stable, red loss, green gain"
)
```

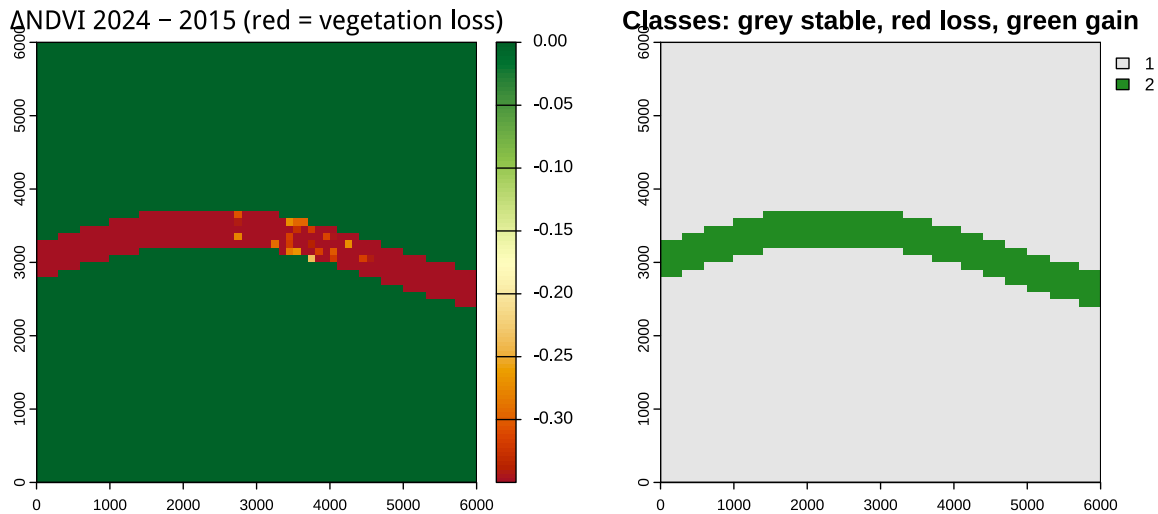


Figure 48: NDVI differencing between two dates: strongly negative = vegetation scoured by the flood.

```
par(mfrow = c(1, 1))
```

How to read the pair of maps:

- **Left:** Continuous NDVI difference. Red = vegetation *lost* (scour, deforestation, urbanisation); green = vegetation *gained* (regrowth, plantations).
- **Right:** The same data classified into three zones — stable (grey), loss (red), gain (green) — using thresholds at ± 0.15 .

The dark-red ribbon traces the **scour zone** along the river — the same area where the channel has migrated. This is how satellite data tells you *where the river has eaten the bank*, without ever visiting the site.

11 One-Page Summary (Pin This!)

Concept	One-line takeaway
Linear regression	Fits a line by least squares — interpret slope as physical sensitivity
Logistic regression	Sigmoid of a linear score gives $P(\text{event})$; odds ratios for interpretation

Concept	One-line takeaway
Decision tree	Yes/no flowchart learned from data; overfits without pruning
Random forest	Committee of decorrelated trees; OOB error is a free test estimate
Variable importance	Which features the forest relies on most; guides factor-map design
Partial dependence	Shape of one feature's effect holding others constant
KNN	Majority vote of nearest neighbours; always scale features; tune k
Naive Bayes	Bayes' theorem + independent-features assumption; fast, transparent
K-means	Partition into K clusters by centroid distance; elbow + silhouette pick K
Hierarchical	Agglomerative tree of merges; dendrogram cut at height gives K
Raster	Grid of cells — DEMs, imagery, rainfall surfaces (terra)
Vector	Points, lines, polygons — pipes, parcels, rivers (sf)
NDVI	$(\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red})$; vegetation health proxy
GIS + AI fusion	<code>extract()</code> map→table, train, <code>predict()</code> table→map
UHI	City centre runs hotter; quantify with LST and NDVI regression
Landslide susceptibility	Logistic/forest model of slope, rain, NDVI, rock → probability map
River change	Overlay two dates; erosion vs deposition areas in km ²

12 Mini Glossary

- **Feature / predictor** — an input column the model learns from (age, rainfall, NDVI).
- **Label / target** — what we predict (strength, flood yes/no, condition class).
- **Sigmoid** — the S-curve $1/(1 + e^{-z})$ that squashes a linear score into $[0, 1]$.
- **Odds ratio** — e^{β} ; multiplicative change in odds per unit increase in a predictor.
- **Gini impurity / entropy** — purity measures used to choose tree splits.
- **Bootstrap sample** — random rows with replacement; the base ingredient of bagging.
- **Bagging** — bootstrap aggregating; trains trees on resampled rows to decorrelate them.
- **OOB (out-of-bag)** — the ~37% of rows each tree never saw; the forest's free validation set.
- **Variable importance** — a score of how much each feature contributes to forest accuracy.
- **Partial dependence** — average model prediction as one feature sweeps its range.
- **Linkage** — the rule defining distance between clusters (single, complete, average, Ward).
- **Dendrogram** — tree diagram of hierarchical merges.
- **Silhouette width** — per-point clustering quality in $[-1, 1]$; near 1 = well placed.
- **Raster / vector** — cell-grid vs. coordinate-geometry spatial data models.
- **CRS** — coordinate reference system; defines what the numbers on the map mean.
- **NDVI** — normalized difference vegetation index;

12.1 References

1. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning: with Applications in R* (2nd ed.). Springer. — Chapters 4, 8, 10, 12 map directly to this unit.
2. Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science* (2nd ed.). O'Reilly.
3. Musakumo, C. (2024). *Explainable Machine Learning for Geospatial Data Analysis: A Data-Centric Approach*. CRC Press. — the susceptibility-mapping workflow of §12-§14.
4. Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
5. Lovelace, R., Nowosad, J., & Muenchow, J. *Geocomputation with R* — free online; the deep reference for `sf` + `terra`.